

Parallel Computing

IT3243

Yasotha Ram Ramanan

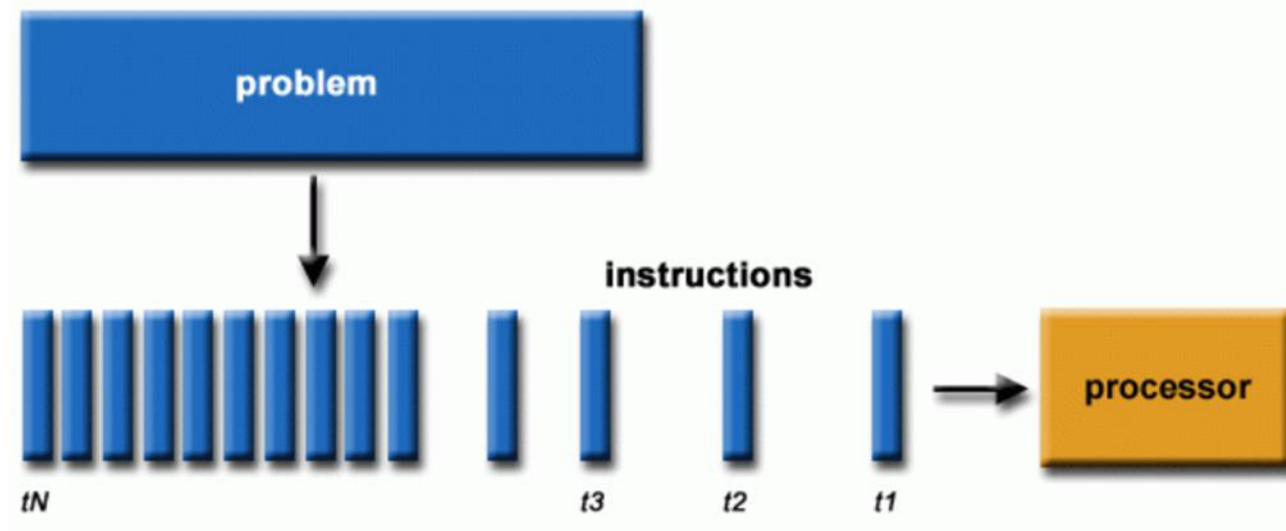
Serial Computing Vs Parallel Computing

Serial Computing

Serial Computing

Traditionally, software has been written for *serial* computation:

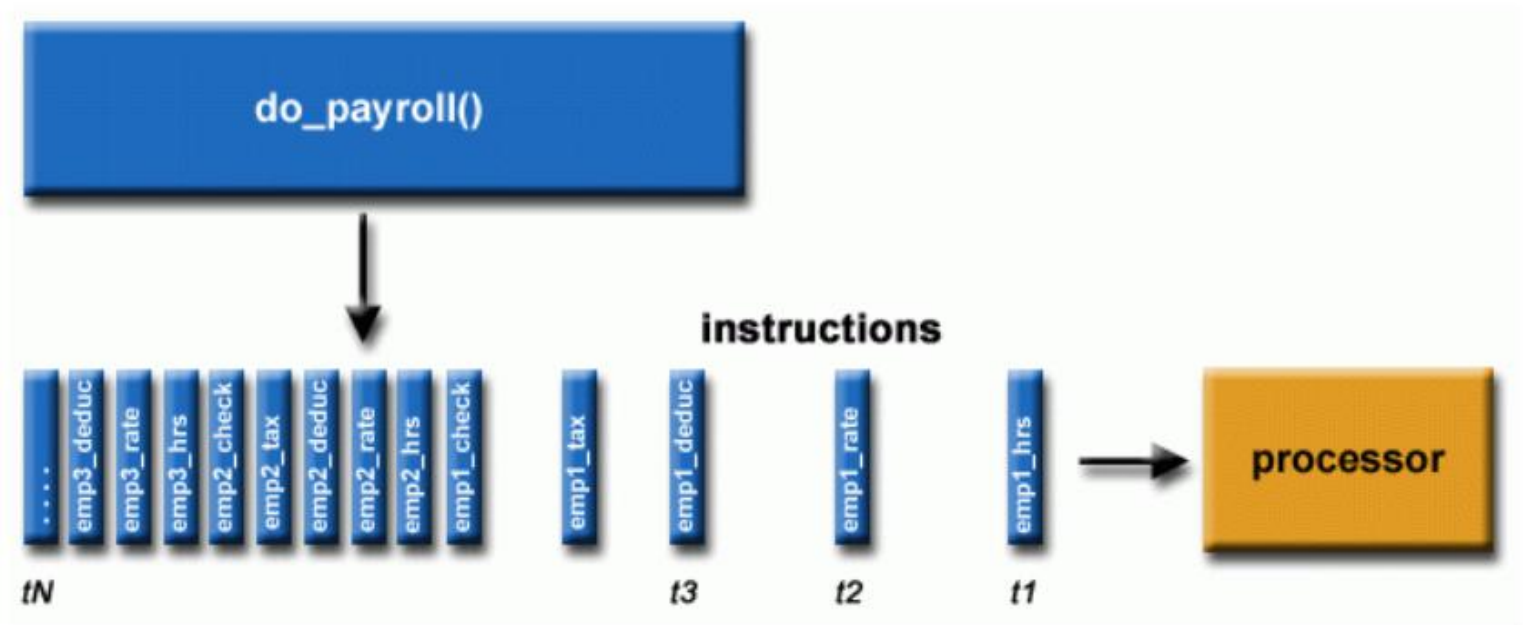
- A problem is broken into a discrete series of instructions
- Instructions are executed sequentially one after another
- Executed on a single processor
- Only one instruction may execute at any moment in time



| Serial computing generic example

Serial Computing ...

For example:



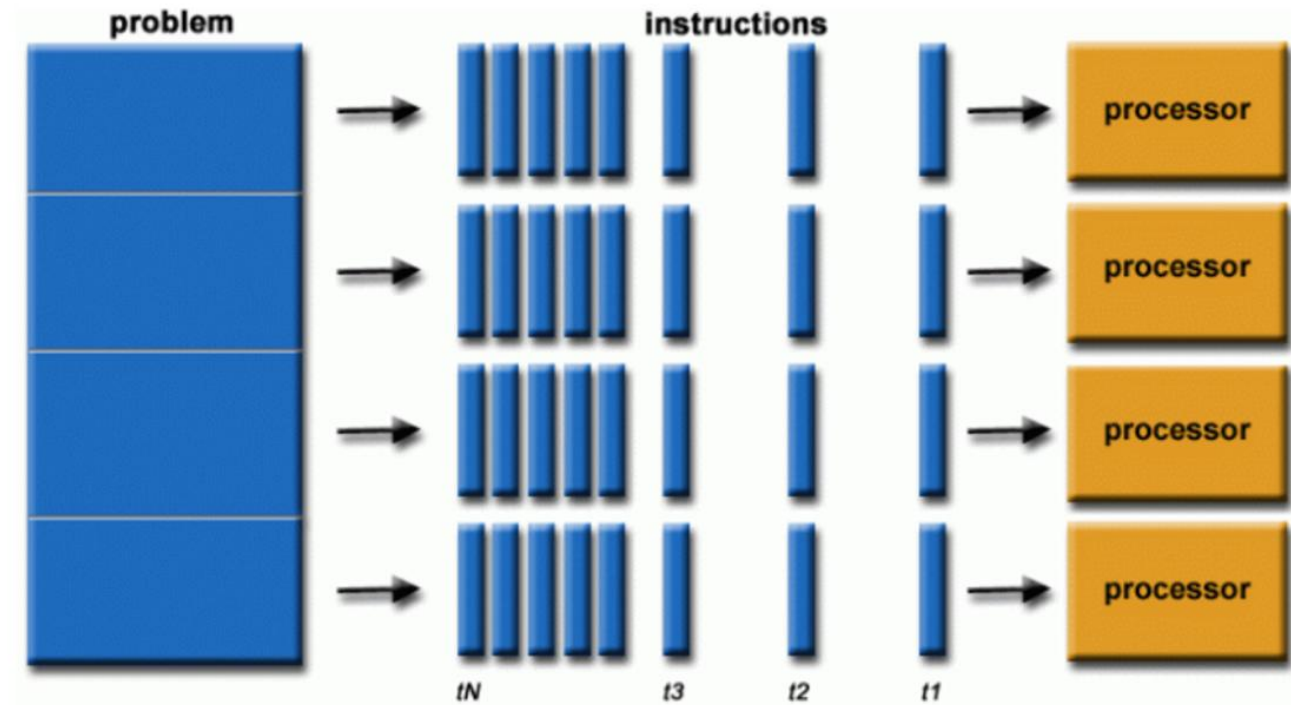
Serial computing example of processing payroll

Parallel Computing

Parallel Computing

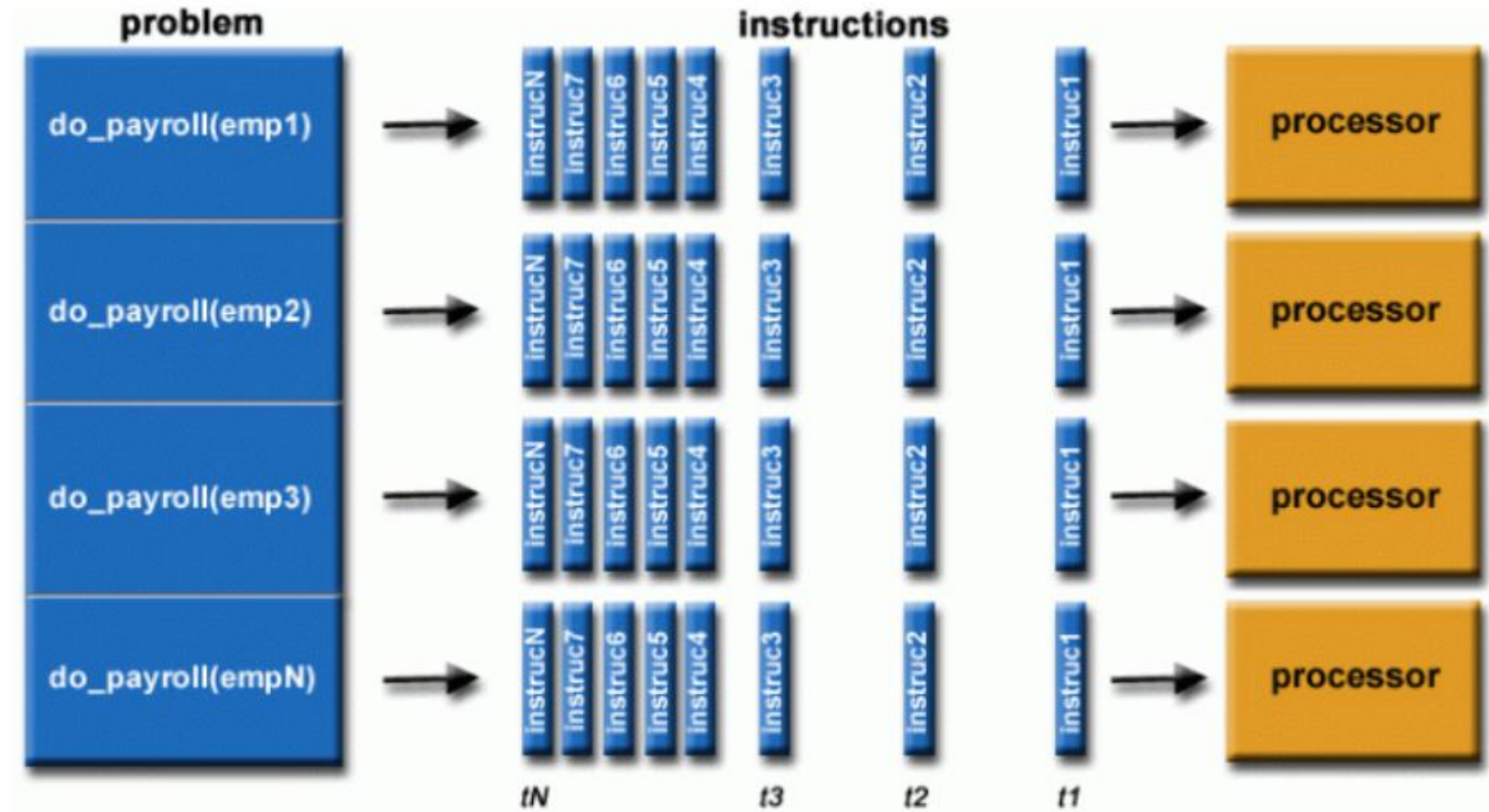
In the simplest sense, *parallel computing* is the simultaneous use of multiple compute resources to solve a computational problem:

- A problem is broken into discrete parts that can be solved concurrently
- Each part is further broken down to a series of instructions
- Instructions from each part execute simultaneously on different processors
- An overall control/coordination mechanism is employed



Parallel Computing

For example:

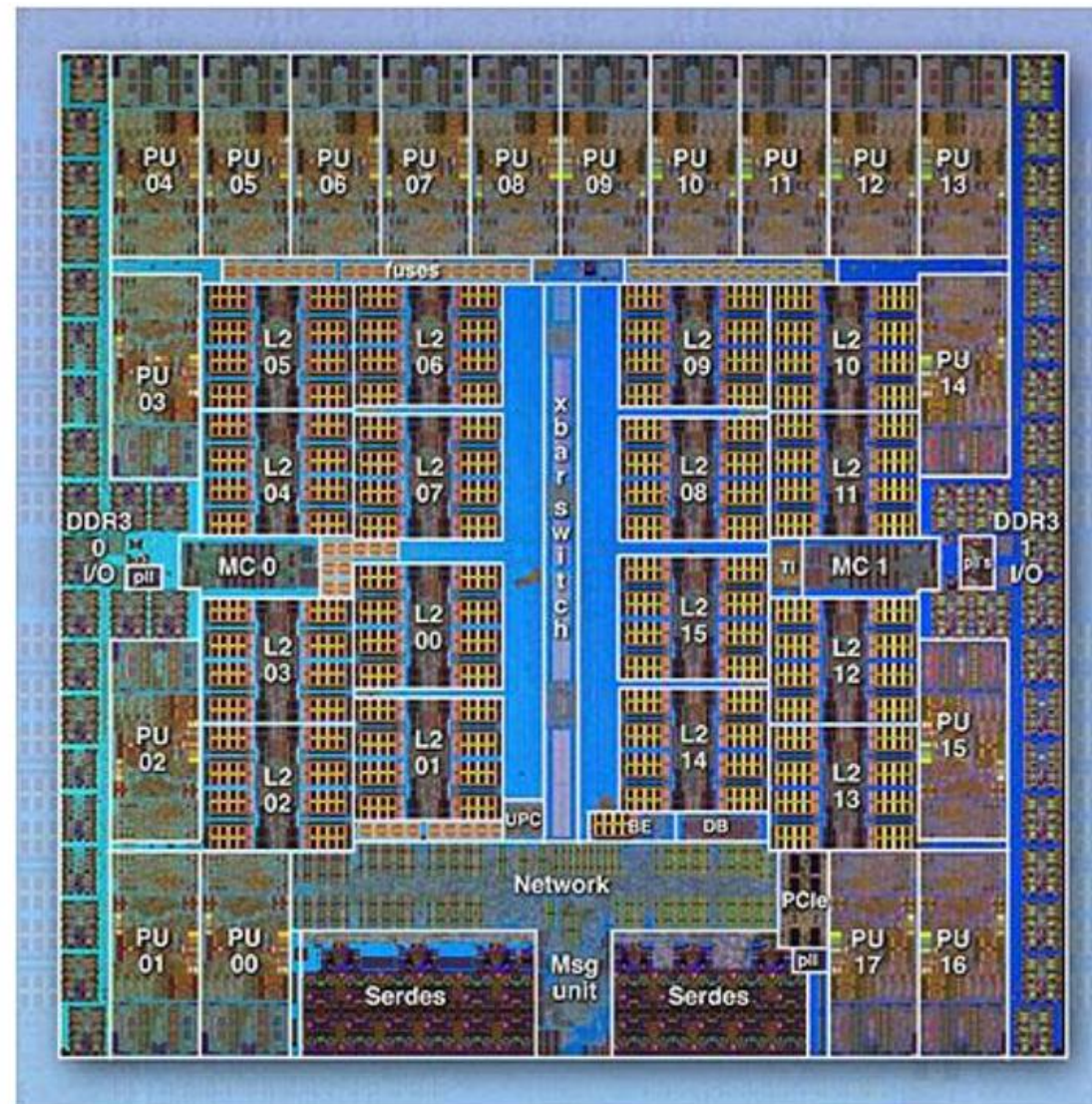


Parallel computing example of processing payroll

- The computational problem should be able to:
 - Be broken apart into discrete pieces of work that can be solved simultaneously;
 - Execute multiple program instructions at any moment in time;
 - Be solved in less time with multiple compute resources than with a single compute resource.
- The compute resources are typically:
 - A single computer with multiple processors/cores
 - An arbitrary number of such computers connected by a network

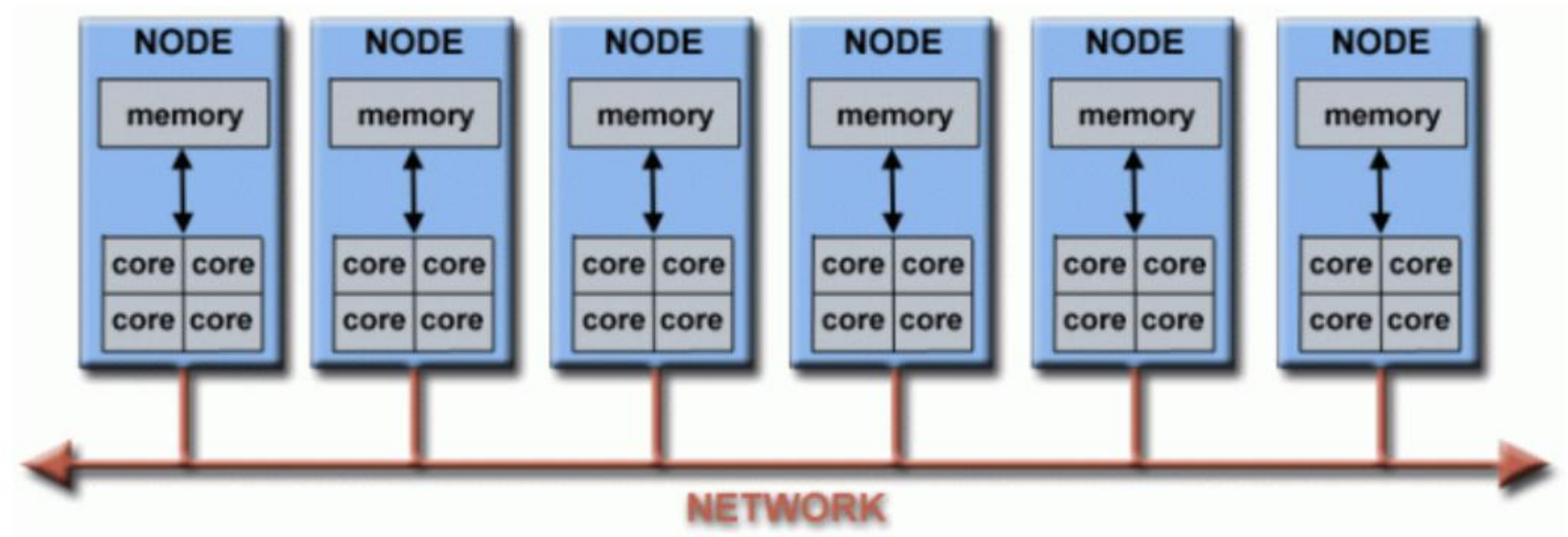
What are Parallel Computers?

- Virtually all stand-alone computers today are parallel from a hardware perspective:
 - Multiple functional units (L1 cache, L2 cache, branch, prefetch, decode, floating-point, graphics processing (GPU), integer, etc.)
 - Multiple execution units/cores
 - Multiple hardware threads



IBM BG/Q Compute Chip with 18 cores (PU) and 16 L2 Cache units (L2)

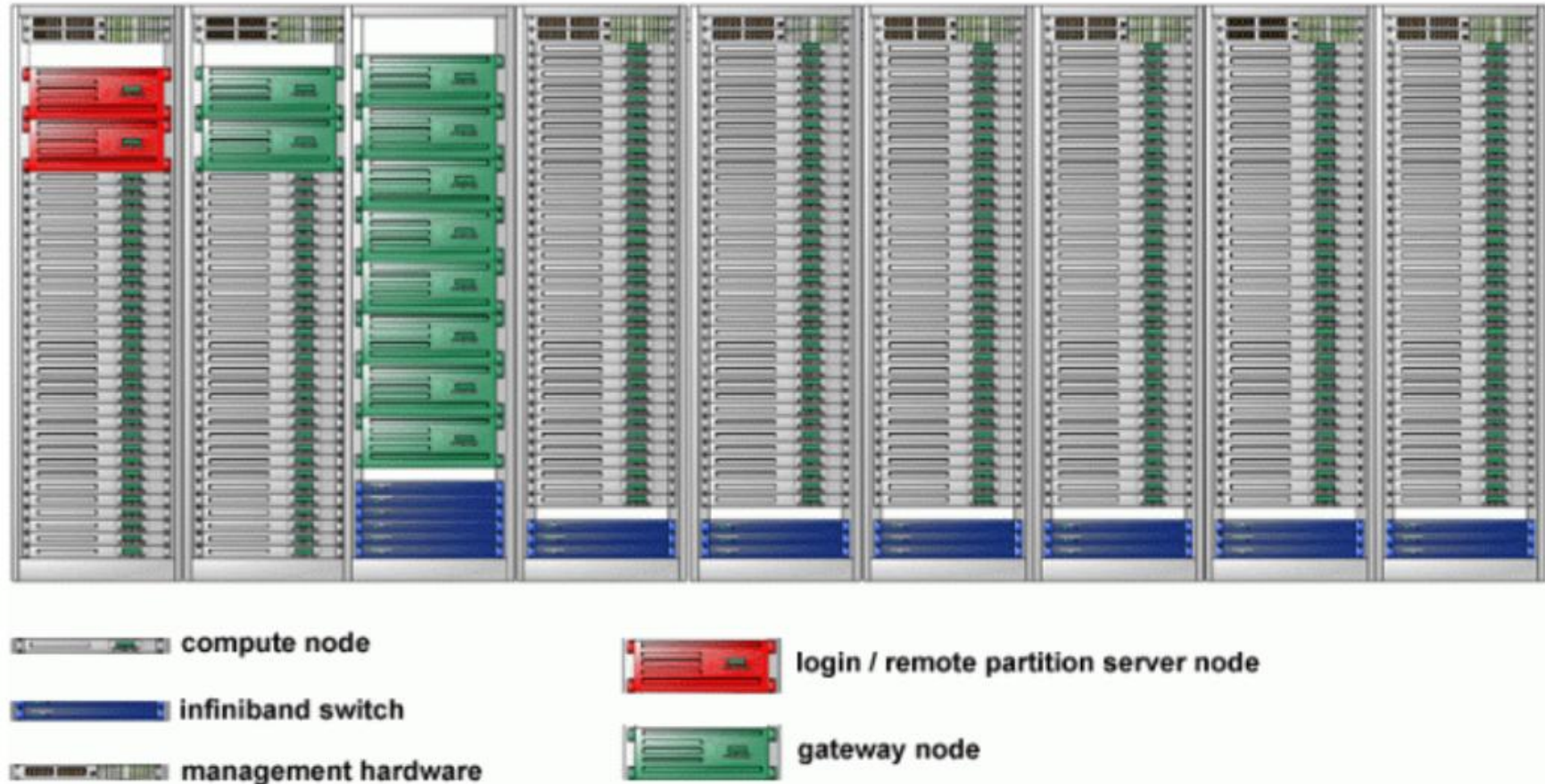
Networks connect multiple stand-alone computers (nodes) to make larger parallel computer clusters.



| *Network connections*

For example, the schematic below shows a typical LLNL parallel computer cluster:

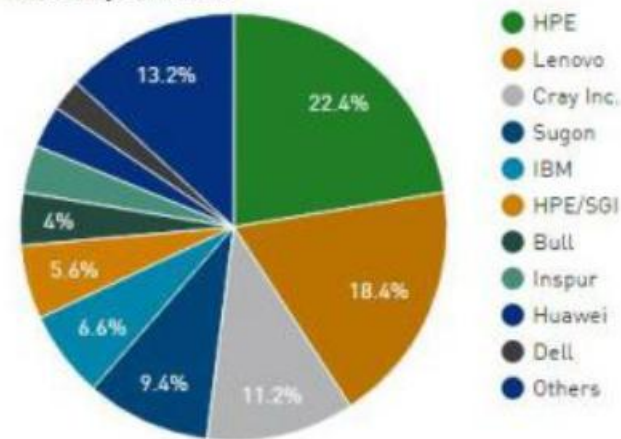
- Each compute node is a multi-processor parallel computer in itself
- Multiple compute nodes are networked together with an Infiniband network
- Special purpose nodes, also multi-processor, are used for other purposes



| *Example of typical parallel computer cluster*

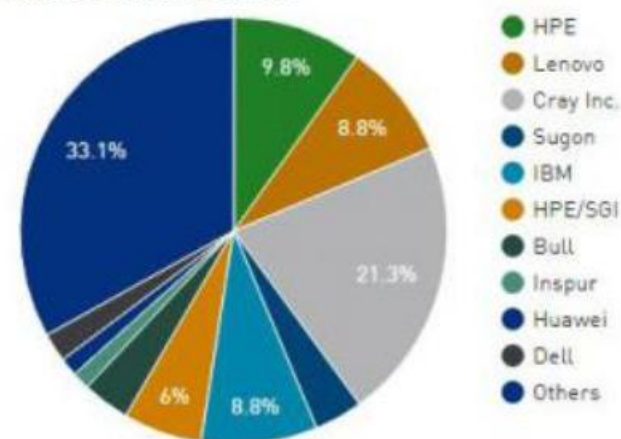
- The majority of the world's large parallel computers (supercomputers) are clusters of hardware produced by a handful of (mostly) well known vendors.

Vendors System Share



Vendors	Count	System Share (%)
HPE	112	22.4
Lenovo	92	18.4
Cray Inc.	56	11.2
Sugon	47	9.4
IBM	33	6.6
HPE/SGI	28	5.6
Bull	20	4
Inspur	18	3.6
Huawei	16	3.2
Dell	12	2.4
Fujitsu	11	2.2
Penguin Computing	9	1.8
NUDT	4	0.8
Lenovo/IBM	4	0.8
NEC	3	0.6
T-Platforms	3	0.6
Atipa	3	0.6
IBM/Lenovo	3	0.6
NRCPC	2	0.4
MEGWARE	2	0.4
RSC Group	2	0.4
Self-made	1	0.2
IPE, Nvidia, Tyan	1	0.2
DALCO	1	0.2
Adtech	1	0.2
ClusterVision	1	0.2

Vendors Performance Share



Why do we use Parallel Computing?

- Save time and/or money
- Solve larger/ more complex problems
- Provide concurrency
- Take advantage of non-local resources
- Make better use of underlying parallel hardware

Parallel Computing Terminology and Parallel Computers Evolutions

A Core

- A CPU core is a [CPU's](#) processor.
- A CPU core is essentially the brain of your computer.
- A CPU core is a single processing unit within the CPU that can execute instructions.
- The more cores a CPU has, the more tasks it can handle simultaneously.
- Intel Core processors

Core i3, Core i5, Core i7

Intel Celeron and Intel Pentium processors.



Microprocessor

- A microprocessor is a computer processor for which the data processing logic and control is included on a single integrated circuit (IC), or a small number of ICs.



Microprocessors ...

- Microprocessors evolved from 4 bit to 64 bit microprocessors, introducing computer technologies such as pipeline, super-pipeline, super-scalar, VLIW, cache memory, and virtual memory system.
- Now, it is possible to integrate 16 sets of microprocessor with 64GB of memory on the board.

Evolution of Intel Microprocessors

1970s Processors

Name of Processor	introduced	Clock speeds	Bus width	Number of Transistors	Addressable Memory
4004	1971	108 KHz	4 Bits	2300	640 Bytes
8008	1972	108 KHz	8 bits	3500	16 KB
8080	1974	2 MHz	8 bits	6000	64KB
8086	1978	5,8,10 MHz	16bits	29000	1MB
8088	1979	5,8 MHz	8 bits	29000	1MB

1980s Processors

Name of Processor	introduced	Clock speeds	Bus width	No. of Transistors	Addressable Memory	Virtual Memory
80286	1982	6MHz-- 12.5 MHz	16 bits	134,000	16 MB	1 GB
386TM DX	1985	16MHz— 33 MHz	32 bits	275,000	4 GB	64 TB
386TM SX	1988	16MHz— 33 MHz	16 bits	275,000	16 MB	64 TB
486TM DX CPU	1989	25MHz— 50 MHz	32 bits	1.2 million	4 GB	64 TB

1990s Processors

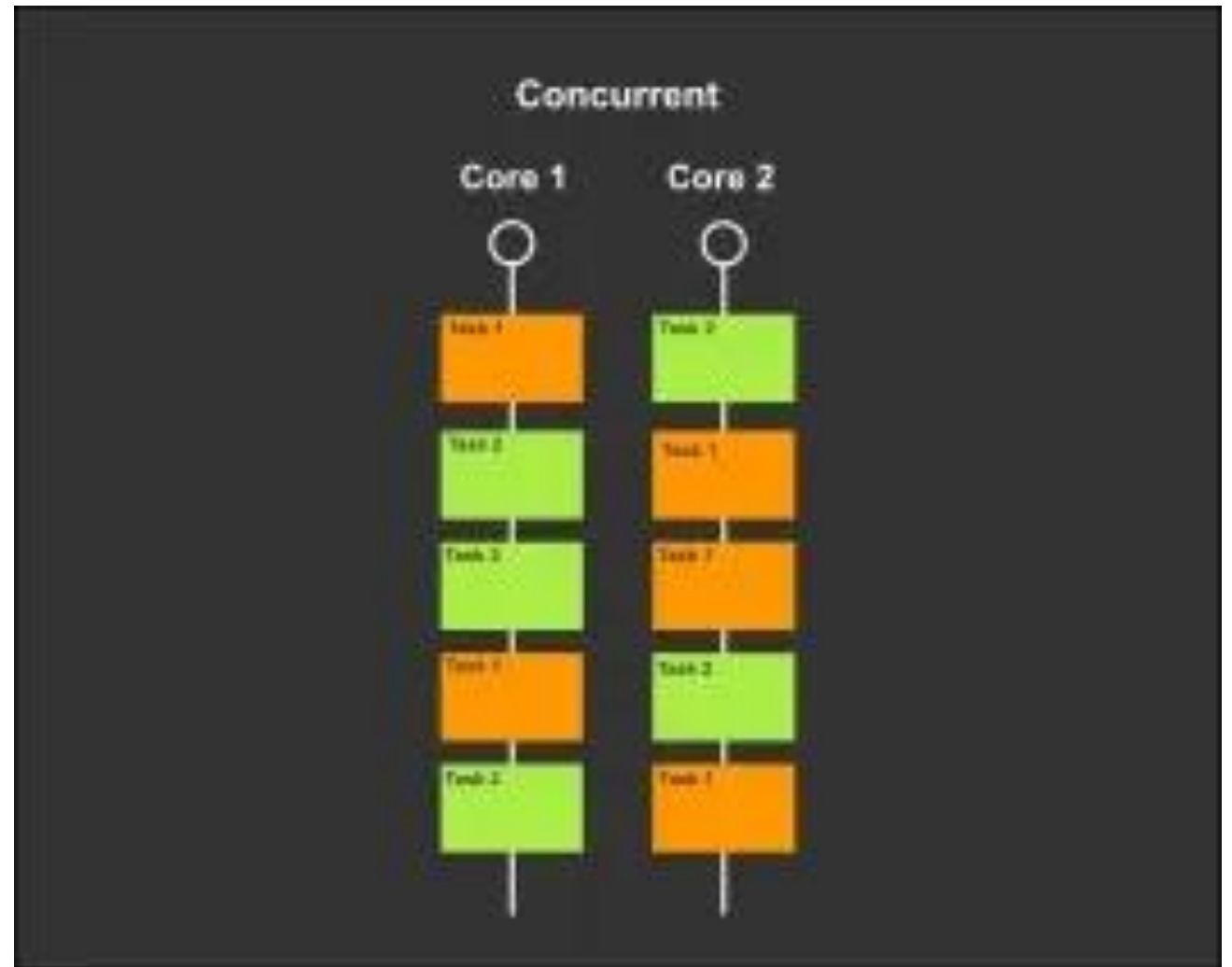
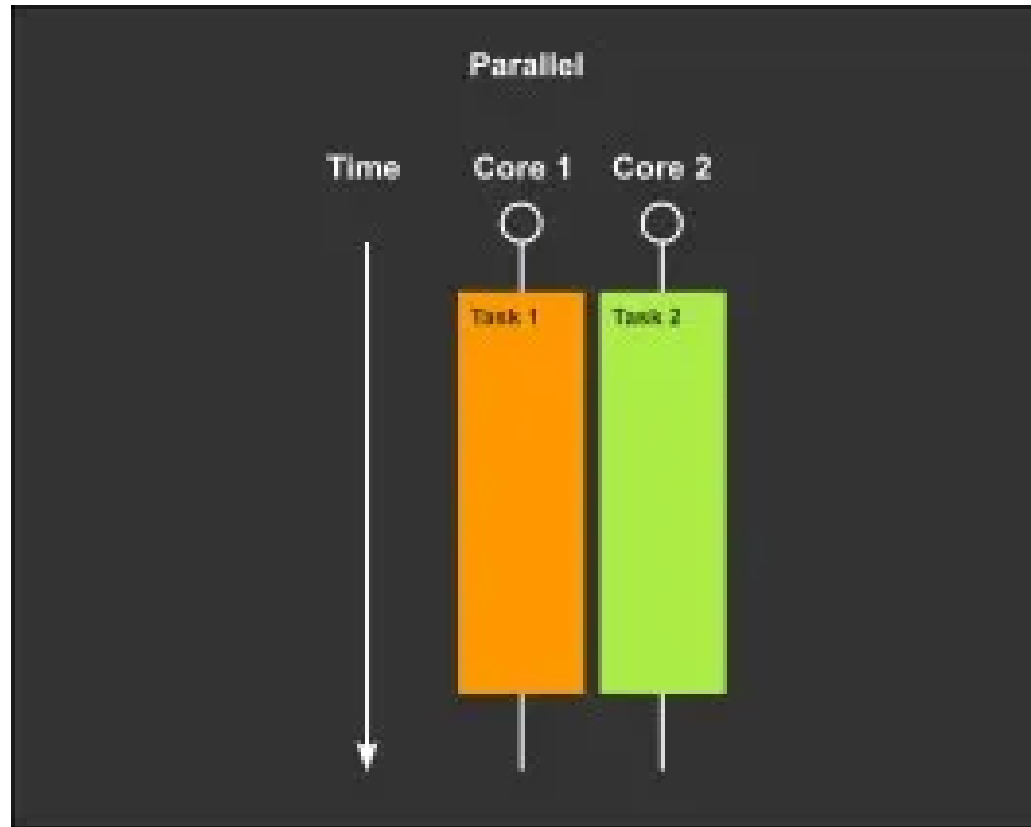
Name of Processor	introduced	Clock speeds	Bus width	No. of Transistors	Addressable Memory	Virtual Memory	Cache
486TM SX	1991	16MHz-33MHz	32 bits	1.185 million	4 GB	64 TB	8 KB
Pentium	1993	60MHz — 166 MHz	32 bits	3.1 million	4 GB	64 TB	8 KB
Pentium Pro	1995	150MHz — 200 MHz	64 Bits	5.5 million	64 MB	64 TB	512 KB & 1 MB
Pentium II	1997	200MHz — 300 MHz	64 Bits	7.5 million	64 GB	64 TB	512KB

Recent Processors

Name of Processor	introduced	Clock speeds	Bus width	No. of Transistors	Addressable Memory	Virtual Memory	Cache
Pentium III	1999	450MHz -- 660MHz	64 Bits	9.5 million	4 GB	64 TB	8 KB
Pentium 4	2000	1.3—1.8 GHz	64 Bits	42 million	4 GB	64 TB	8 KB
Core 2 Duo	2006	1.06—1.2 Ghz	64 Bits	167 million	64 MB	64 TB	512 KB & 1 MB
Core 2 Quad	2008	3 Ghz	64 Bits	820 million	64 GB	64 TB	512KB

Concurrent VS. Parallelism

Parallelism



Concurrent VS. Parallelism

- **Concurrency** focuses on managing multiple tasks seemingly at the same time, while parallelism involves executing multiple tasks truly simultaneously using multiple processing units
- Parallelism is the ability to execute **independent tasks** of a **program** in the **same instant of time**.
- Contrary to concurrent tasks, these tasks can run simultaneously on a single processor, or an entirely different computer that can be a distributed system.
- **Parallelism** happens **on at least two processors**

Concurrency Vs Parallel

Concurrency

Making progress on more than one task - seemingly at the same time.



Parallel

Parallel Execution

Making progress on more than one task at the exact same time.



Parallel Concurrent Execution

Parallel Concurrent Execution

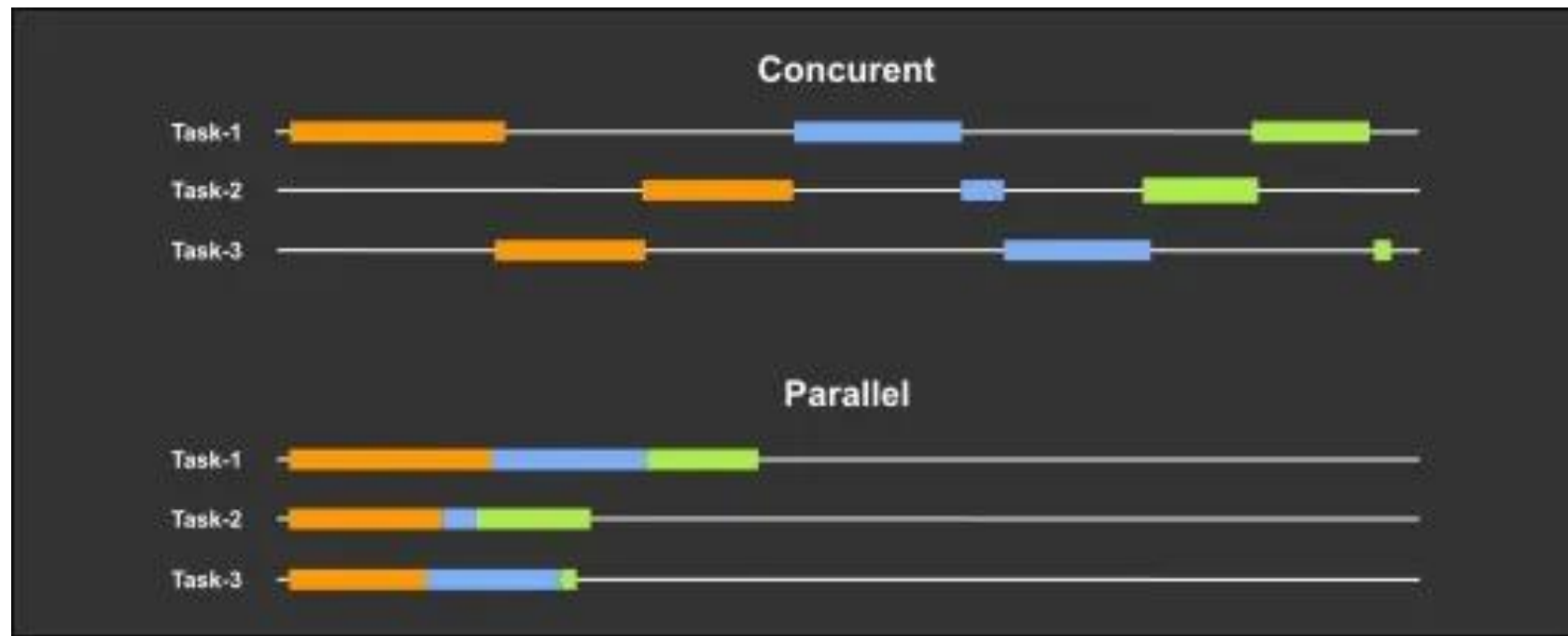
Making progress on more than one task - seemingly at the same time - on more than one CPU.



Parallelism

Parallelism

Splitting a single task into subtasks which can be executed in parallel.



Concurrency

Vs

Parallelism

Illusion of parallelism, where only one task is performed at a time.

Meaning

Quality of occurring at the same time.

Interleaving

Achieved through

Using multiple processors.

Context Switching

Uses

Multiple CPU's for operating multiple processes.

Single

No of processing units required

Multiple

Types of Parallelism

Types of Parallelism

Types of Thread-Level Parallelism

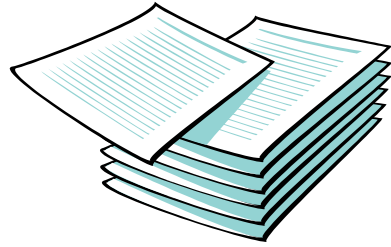
Given below are three types of Thread level parallelism:

- **Instruction-Level Parallelism (ILP):** This form of TLP focuses on executing multiple instructions from a single thread in parallel. Techniques such as pipelining and superscalar architectures fall under this category.
- **Data-Level Parallelism (DLP):** DLP involves executing the same operation on multiple data elements simultaneously, often seen in SIMD (Single Instruction, Multiple Data) architectures.
- **Task-Level Parallelism (TLP):** TLP refers to executing multiple independent threads concurrently. This is particularly relevant in today's context, as it aligns with the trend of increasing processor core counts.

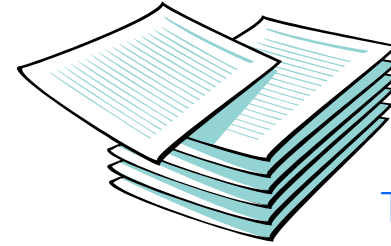
Data Parallelism

Division of work – data parallelism

TA#1

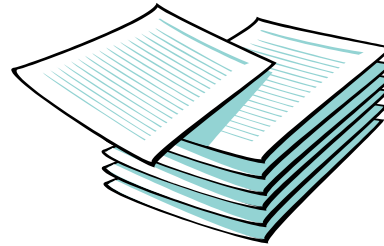


100 exams



TA#3

100 exams



TA#2

100 exams

Task Parallelism

Division of work – task parallelism

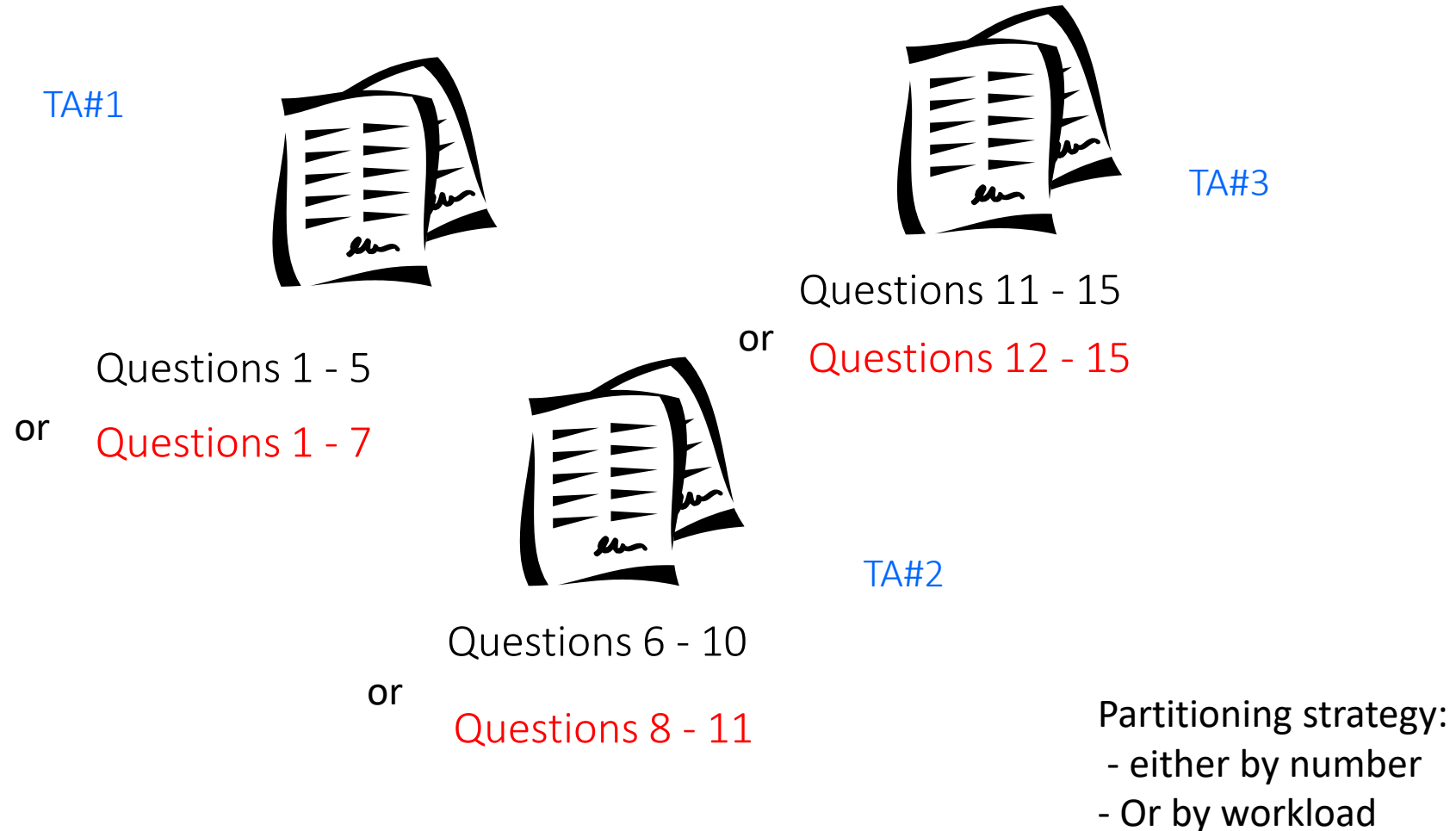


Figure 1.1 Everyday parallelism in supermarket checkout queues. The checkout cashiers (with caps) process their queue of customers (with baskets). On the left, one cashier processes four self-checkout lanes simultaneously. On the right, one cashier is required for each checkout lane. Each option impacts the supermarket's costs and checkout rates.

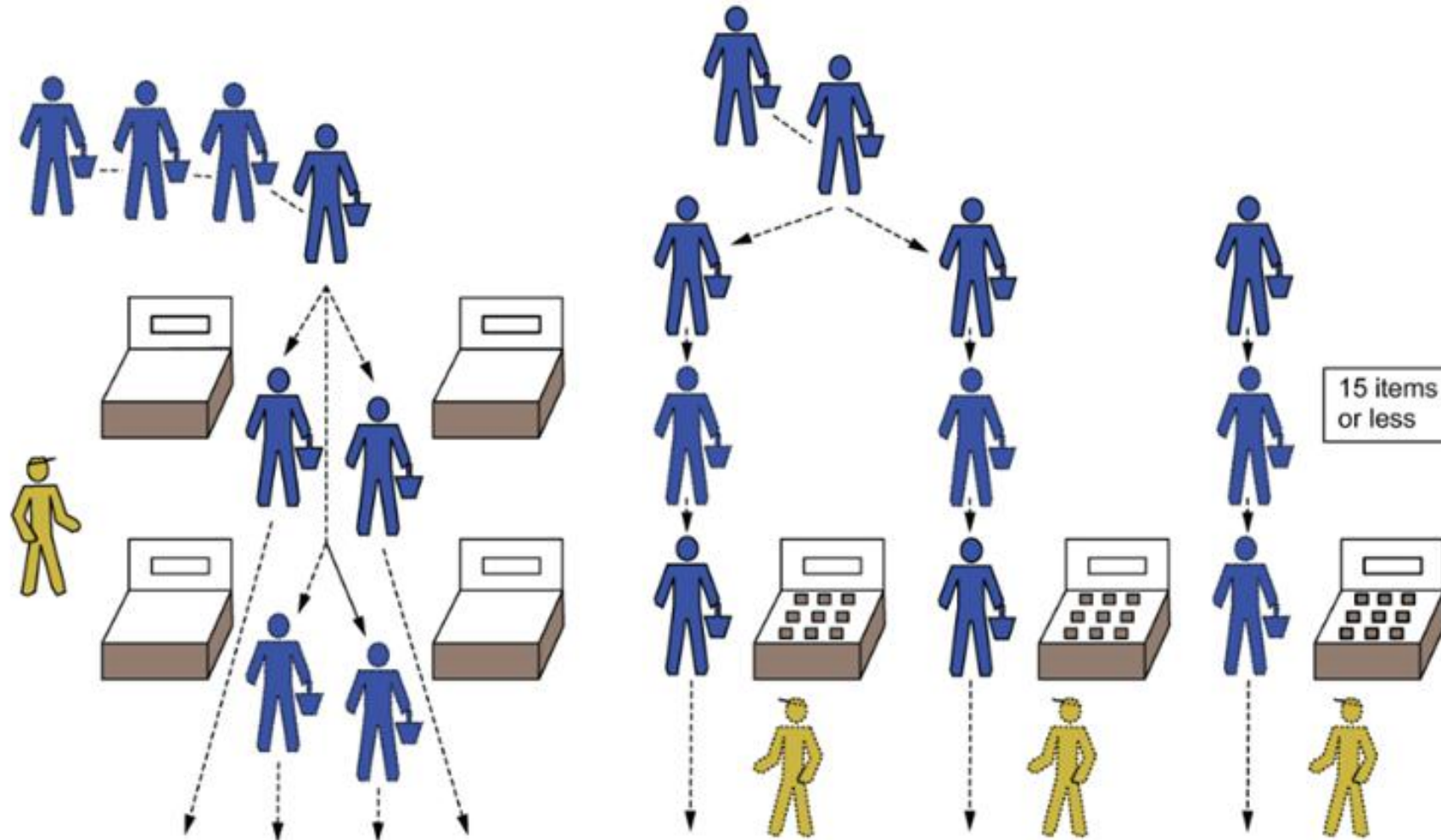
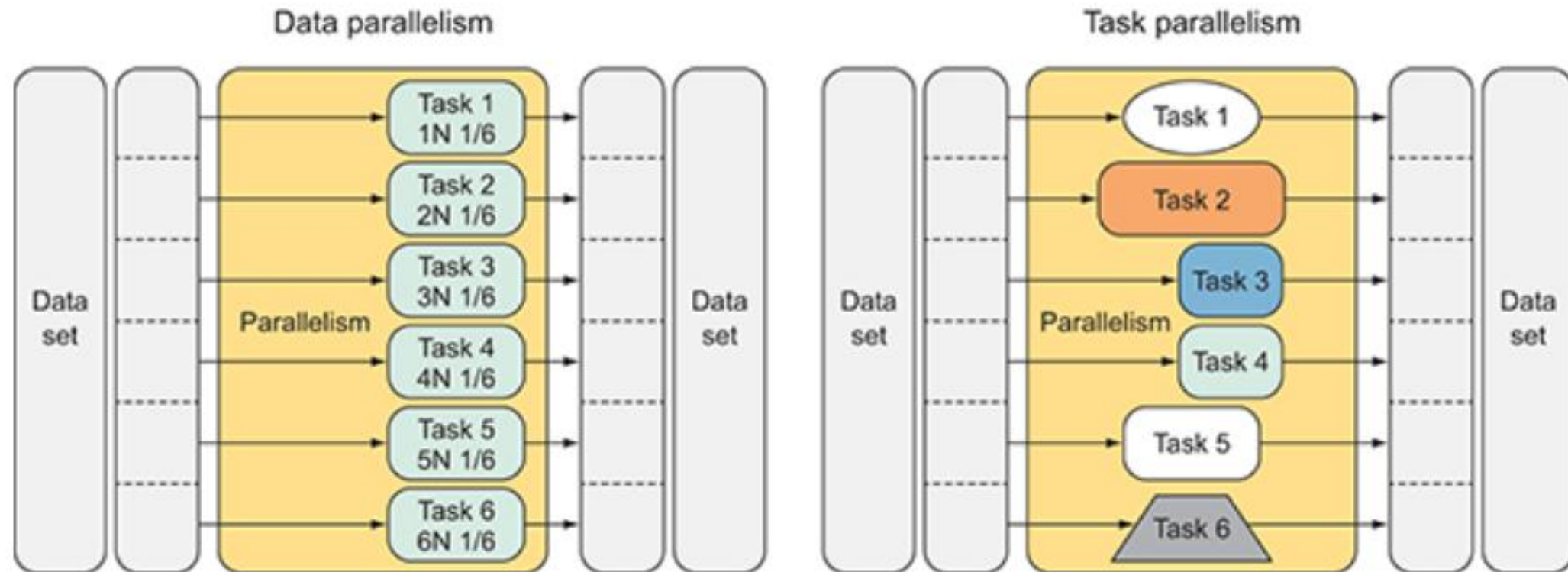


Figure 7.1 Data parallelism is the simultaneous execution of the same function across the elements of a data set. Task parallelism is the simultaneous execution of multiple and different functions across the same or different data sets.



- An application can be concurrent — but not parallel, which means that it processes more than one task at the same time, but no two tasks are executing at the same time instant.
- An application can be parallel — but not concurrent, which means that it processes multiple sub-tasks of a task in multi-core CPU at the same time.
- An application can be neither parallel — nor concurrent, which means that it processes all tasks one at a time, sequentially.
- An application can be both parallel — and concurrent, which means that it processes multiple tasks concurrently in multi-core CPU at the same time.

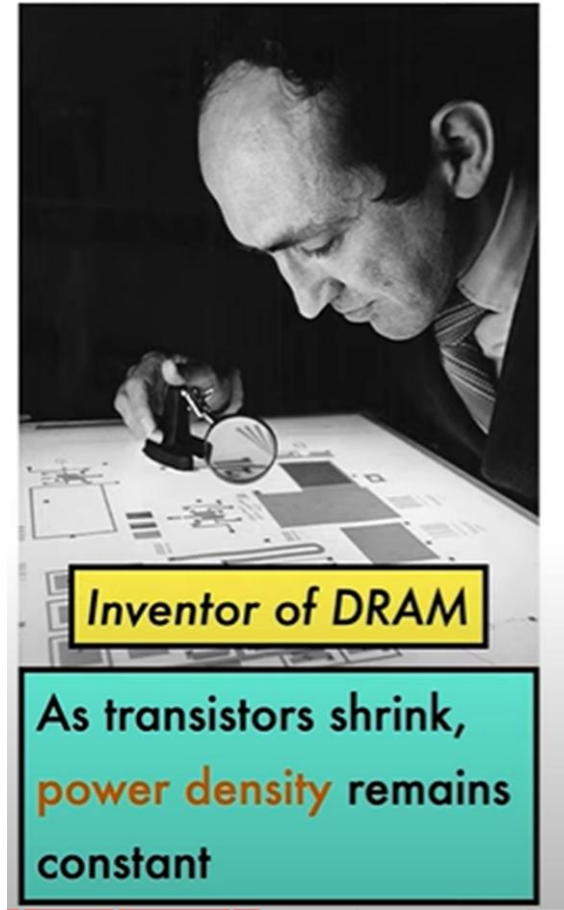
Exploit parallelism

Mechanisms to Exploit Thread level parallelism(TLP)

Mechanisms to exploit Thread level parallelism(TLP) are given below:

- **Multicore Processors:** One of the most tangible embodiments of TLP is the advent of multicore processors. These processors feature multiple independent processing cores on a single chip, each capable of executing threads in parallel.
- **Simultaneous Multithreading (SMT):** SMT, often referred to as Hyper-Threading, allows a single physical core to execute multiple threads simultaneously, effectively increasing core-level thread-level parallelism.
- **Task Scheduling and Load Balancing:** Efficient thread scheduling and load balancing algorithms ensure that tasks are distributed across available cores optimally, maximizing resource utilization.

Moore's law, Dennard Scaling, and Amdahl's Law



Moore's law

- **Moore's law** is the observation that the number of transistors in an [integrated circuit](#) (IC) doubles about every two years.
- Moore's law is an [observation](#) and [projection](#) of a historical trend. Rather than a [law of physics](#), it is an [empirical relationship](#) linked to [gains from experience](#) in production.

Moore's law

- The observation is named after [Gordon Moore](#), the co-founder of [Fairchild Semiconductor](#) and [Intel](#) (and former CEO of the latter), who in 1965 posited a [doubling every year](#) in the number of components per integrated circuit,^[a] and projected this rate of growth would continue for at least another decade.
- In 1975, looking forward to the next decade, he revised the forecast to doubling every two years, a [compound annual growth rate](#) (CAGR) of 41%.
- While Moore did not use empirical evidence in forecasting that the historical trend would continue, his prediction has held since 1975 and has since become known as a "law".

How Has Moore's Law Impacted Computing?

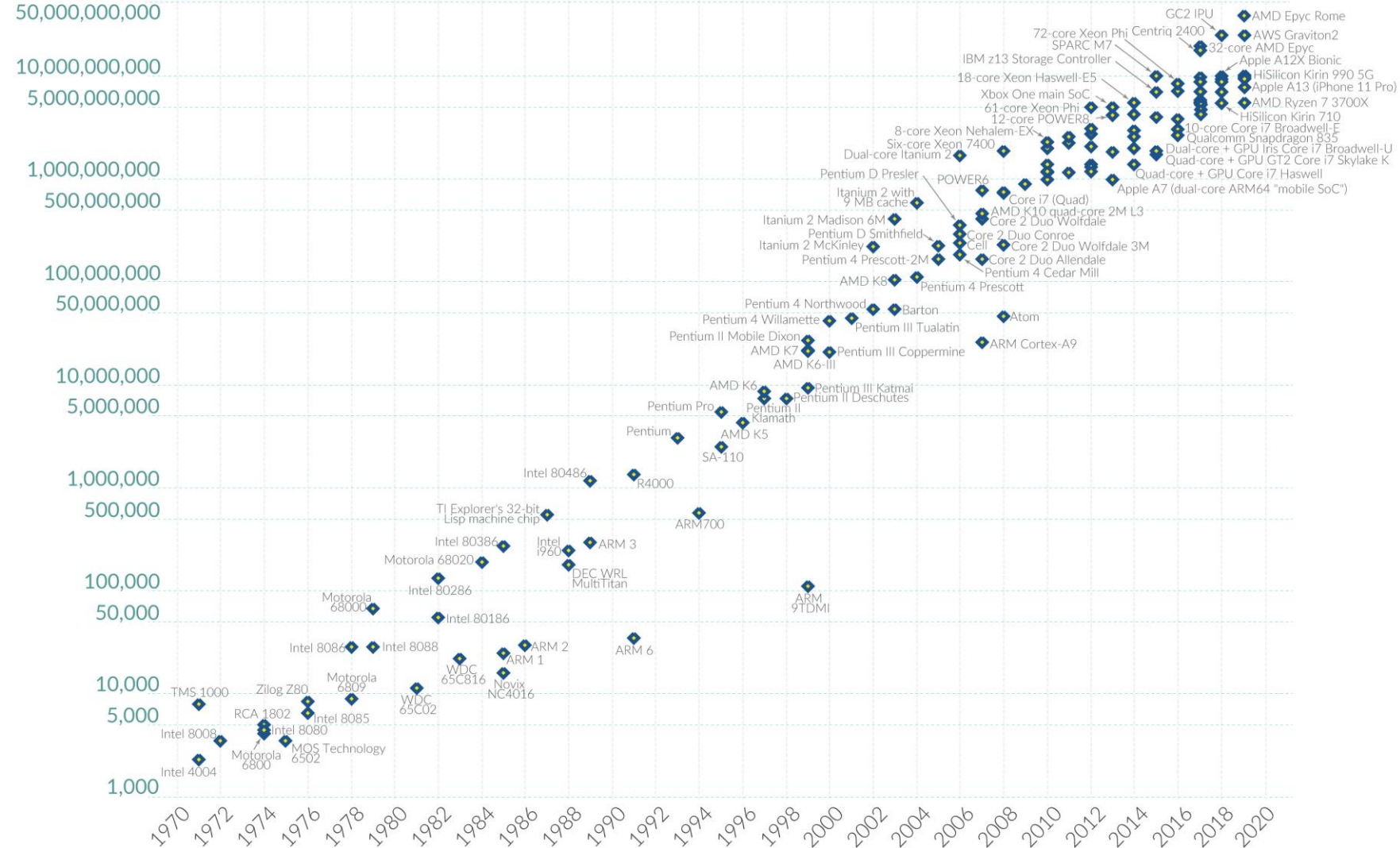
- Moore's Law has directly influenced the progress of computing power by creating a goal for chip makers to achieve. In 1965, Moore predicted that there would be 65,000 transistors per chip by 1975. In 2024, chip makers can put 50 billion transistors on a chip the size of a fingernail
- Moore's Law implies that computers, machines that run on computers, and computing power all become smaller, faster, and cheaper with time as processes become more efficient and components smaller and faster.

Moore's Law: The number of transistors on microchips doubles every two years



Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

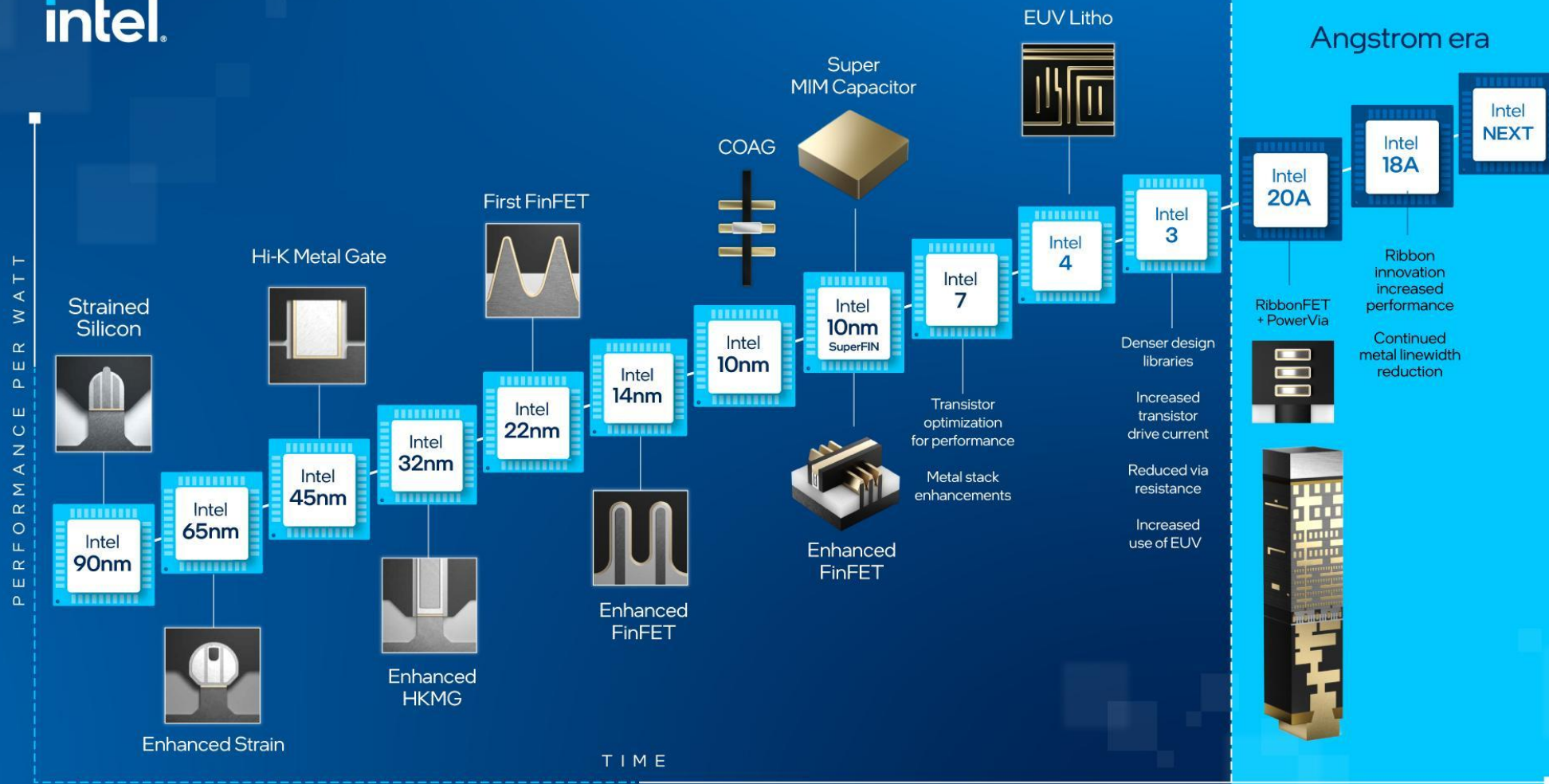
Transistor count



Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.



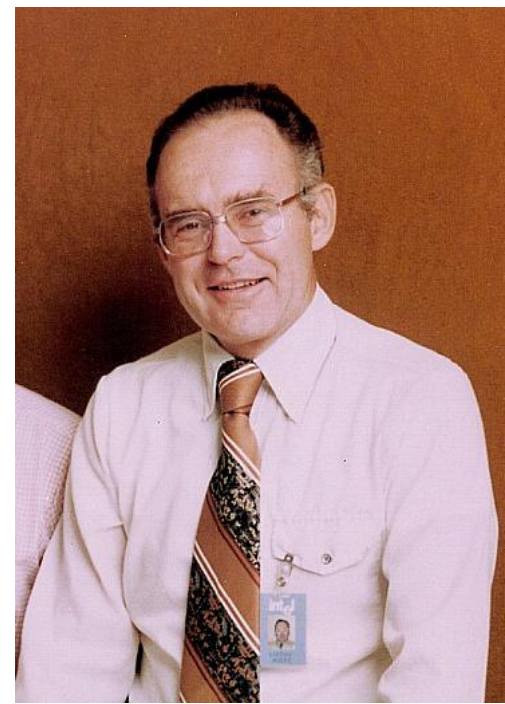
Trends in Microprocessors

- **Moore's Law**
- **Dennard Scaling**
- **Amdahl's Law**

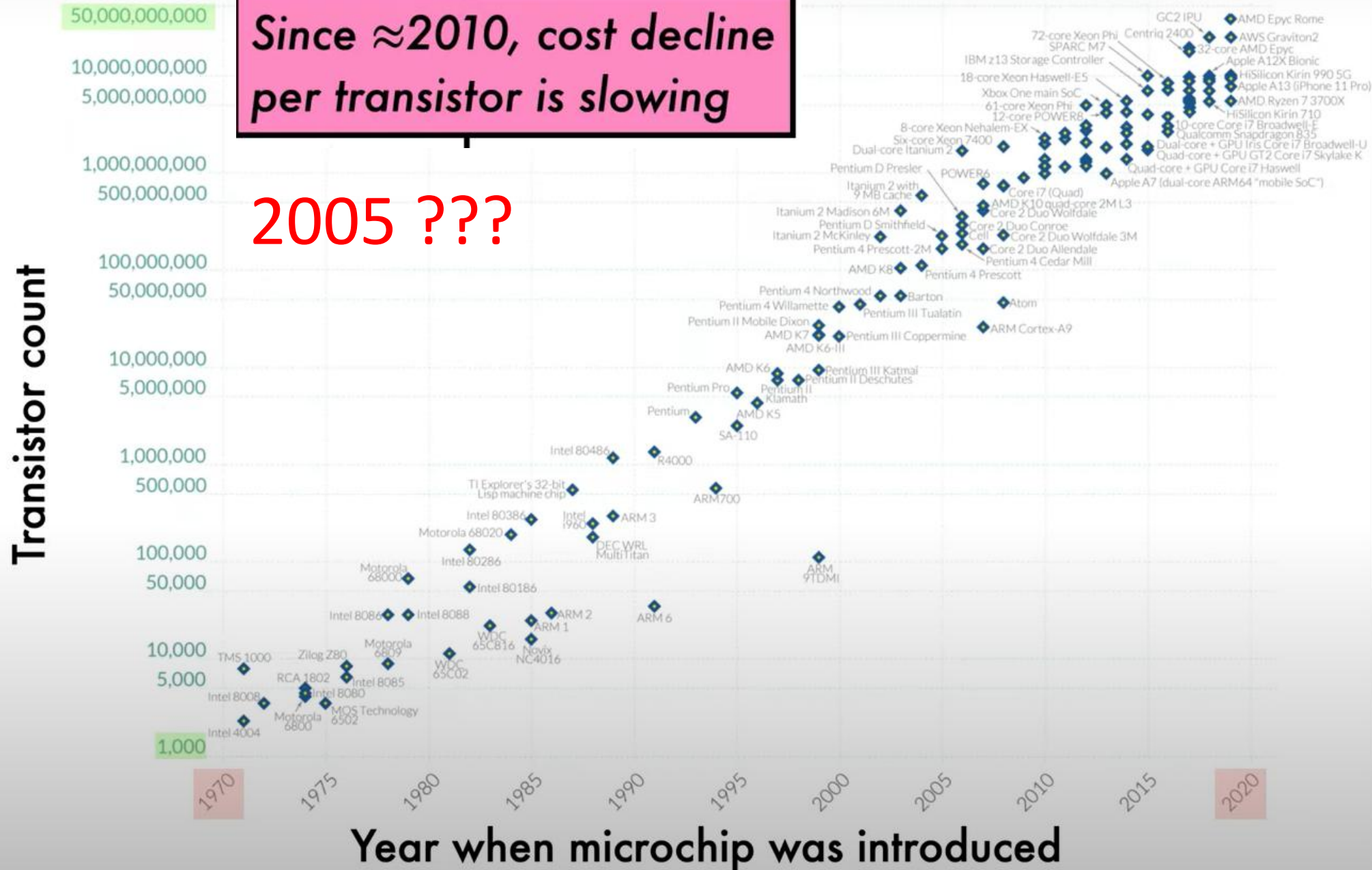
Moore

Moore's Law: The Engine of Progress

- Gordon Moore
- [co-founder](#) and emeritus chairman of [Intel Corporation](#)
- **Moore's law** is the observation that the number of transistors in an [integrated circuit](#) (IC) doubles about every two years.
- Moore's Law posits that the number of transistors on a microchip doubles approximately every two years, leading to exponential increases in **computing power**.
- Moore's Law and single processor performance correlated well-size and speed are related



2005 ???



Moore's Law - 2005

Transistors
Per Die

10^{10}

10^9

10^8

10^7

10^6

10^5

10^4

10^3

10^2

10^1

10^0

1960

1965

1970

1975

1980

1985

1990

1995

2000

2005

2010

1K

4K

16K

64K

256K

1M

4M

16M

64M

128M

256M

512M

1G

2G

4004

8008

8080

8086

80286

386™ Processor

486™ Processor

Pentium® Processor

Pentium® II Processor

Pentium® III Processor

Pentium® 4 Processor

Itanium™ Processor

Itanium™ 2 Processor

◆ 1965 Data (Moore)

■ Memory

◆ Microprocessor

Source: Intel

- In accordance with Moore's Law, the evolution of microprocessors has seen a consistent increase in transistor density.
- For instance, **consider Intel's processor development.** The transition from the 22nm process technology in the 4th generation Intel Core processors to the 14nm process in the 6th generation represents a stride in accordance with Moore's Law, enabling more transistors and improved performance within a similar chip size.

Dennard Scaling

as transistors are reduced in size, their power density stays constant.





Dennard's model of MOSFET scaling

With each generation, transistor dimensions **shrink** by 30%

Device area shrinks by 50%

area = length $\times$ width

Capacitance shrinks by 30%

capacitance $\propto$ area/distance

Voltage is reduced by 30%

electric field $\propto$ voltage/distance

Circuit delay reduces by 30%

due to reduced gate delays

Frequency increases by 40%

frequency = 1/time period

Active power reduces by 50%

active power $\propto CV^2f$

Each generation:

double # transistors

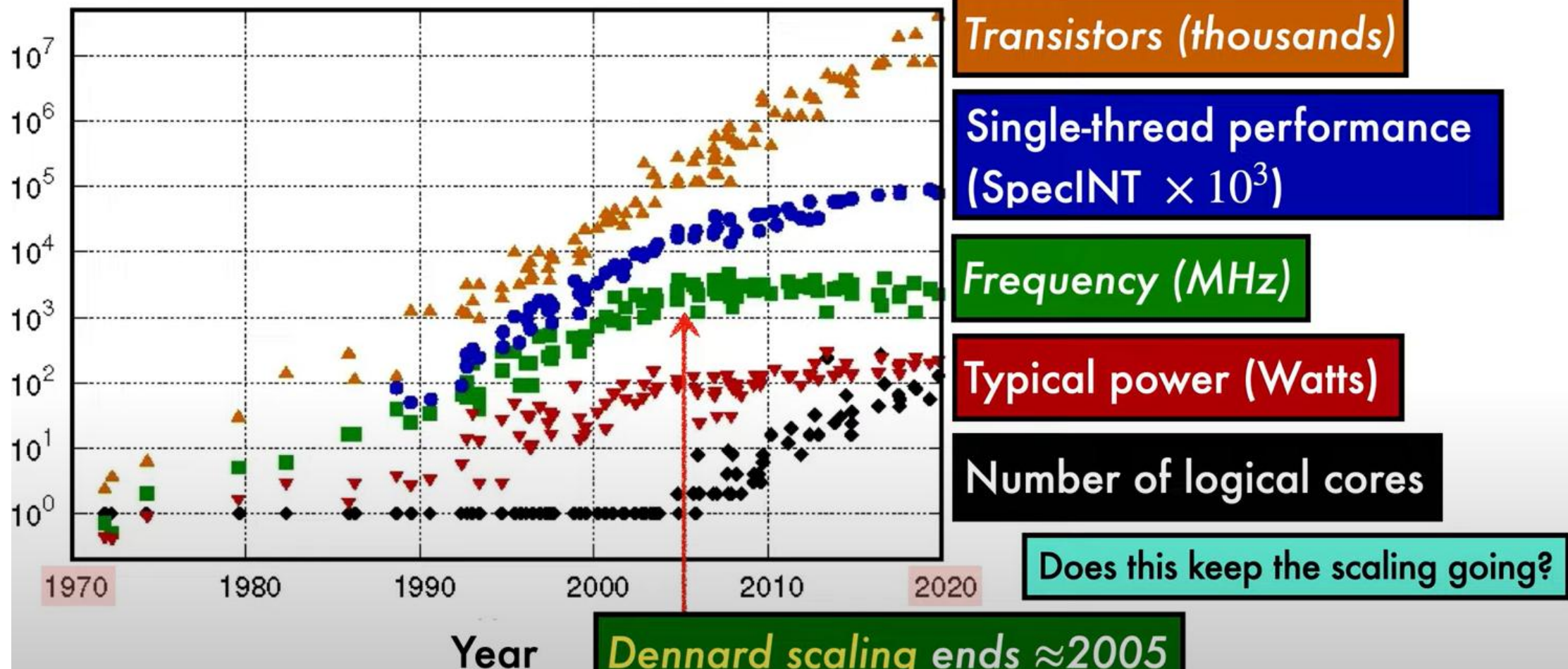
same power

40% faster

DRAM

- DRAM (dynamic random access memory) is a type of semiconductor memory that is typically used for the data or program code needed by a computer processor to function

Microprocessor trends



rocessor-trend-data

er al., data from 2010-2019 by K. Rupp

OSFET scaling paper". IEEE Solid-State Circuits Society Newsletter (2007)

Dennard scaling/ Dennard's Law

- In [semiconductor electronics](#), **Dennard scaling**, also known as **MOSFET scaling**.
- Dennard scaling describes the relationship between transistor size reduction, power density, and performance
- **Dennard scaling** is a [scaling law](#) which states roughly that, as [transistors](#) get smaller, their [power density](#) stays constant, so that the power use stays in proportion with area; both [voltage](#) and [current](#) scale (downward) with length.
- The law, originally formulated for [MOSFETs](#), is based on a 1974 paper co-authored by [Robert H. Dennard](#), after whom it is named

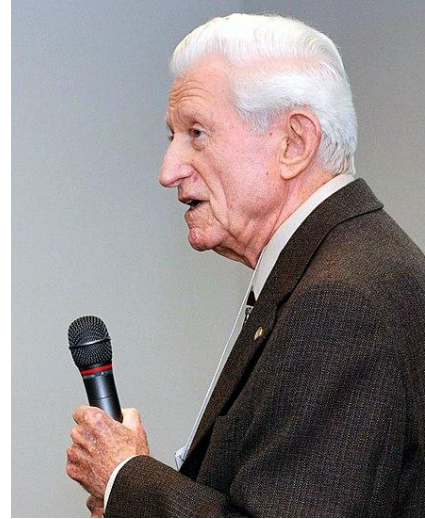
Dennard's model of MOSFET scaling implies that, with every technology generation:

- Transistor dimensions could be scaled by -30% ($0.7\times$). This has the following effects simultaneously:
 - The area of an individual device reduces by 51%, because [area is length times width](#).
 - The capacitance associated with the device, C , is reduced by 30% ($0.7\times$), because [capacitance varies with area over distance](#).
 - To keep the electric field unchanged, the voltage, V , is reduced by 30% ($0.7\times$), because [voltage is field times length](#).
 - Characteristics such as current and transition time are likewise scaled down by 30%, due to their [relationship with capacitance and voltage](#).
 - Overall circuit delay is assumed to be dominated by transition time, so it too is reduced by 30%.
 - Frequency f can increase by about 40% ($1.4\times$), because [frequency varies with one over delay](#).
- Power consumption of an individual transistor decreases by 51%, because [active power](#) is CV^2f .
- As a result, power consumption per unit area remains the same for every technology generation. Alternatively, with every generation the number of transistors in a chip can be doubled with no change in power consumption.

Moore's Law & Dennard scaling/ Dennard's Law

- Dennard's contribution to technology lies in his work on scaling down transistor sizes while maintaining their performance characteristics. He developed the idea that as transistors are made smaller, their voltage and current could also be scaled proportionally, resulting in increased transistor density and improved performance without a significant increase in power consumption. This concept has been instrumental in driving the advancements in semiconductor technology and enabling the continuous improvement of computing devices in accordance with Moore's Law

Amdahl's Law: Optimizing Parallel Processing



Amdahl's law ... *Amdahl's argument*...

- Gene Amdahl - a computer architect from IBM and Amdahl corporation
- Presented at the AFIPS Spring Joint Computer Conference in 1967.
- Addresses **the balance between parallel and sequential processing**.
- Amdahl's Law states that the **speedup achievable by parallelizing a task is limited by the fraction of the task that cannot be parallelized**
- It is a formula that gives the theoretical **speedup** in latency of the execution of a task at a fixed workload that can be expected of a system whose resources are improved.
- In semiconductor design, especially in the development of multi-core processors, this law underscores the importance of optimizing parallelization for tasks that can benefit from it.
- For E.g.... for instance, Gaming processors must carefully balance parallel and sequential processing to achieve optimal performance gains..

- Program
 - A part which cannot be parallelized
 - A part which can be parallelized.

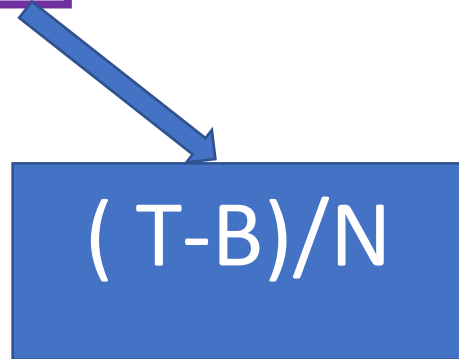
T = total time of serial execution

B = total time of non-parallelizable part

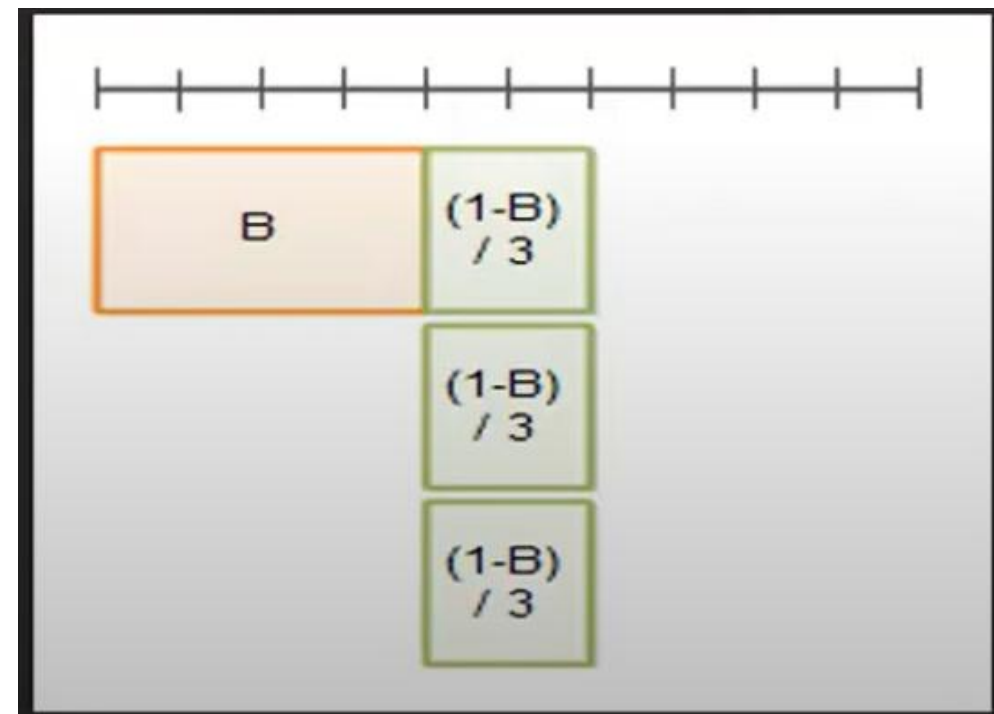
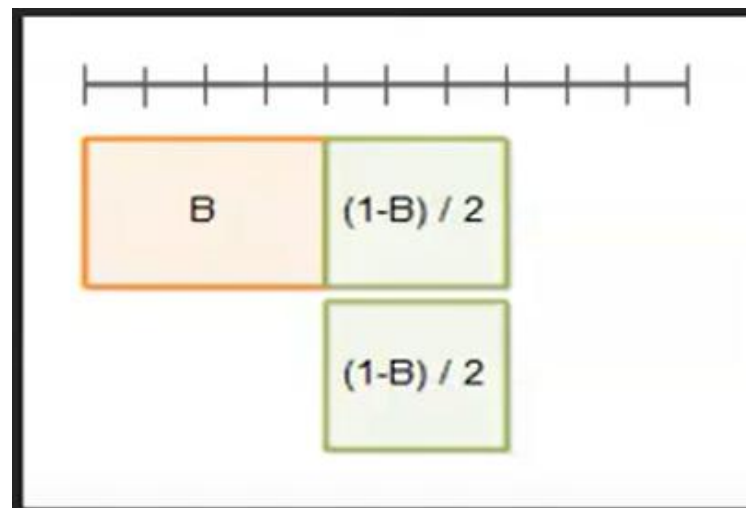
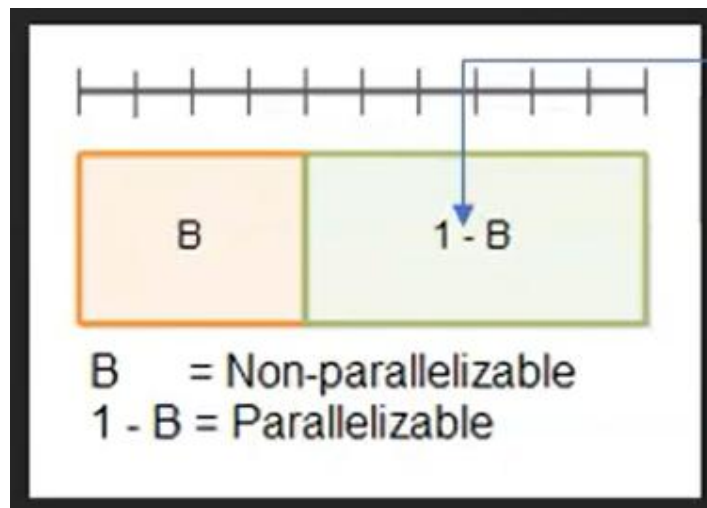
$T - B$ = total time of parallelizable part (When executed serially, not in parallel)

N = the number of threads or CPUs

$$T = B + (T - B)$$


$$(T - B) / N$$

Speed up



$$T(N) = B + (T - B) / N$$

$$T(N) = B + (T - B) / N$$

Example 1:

$$T = 1$$

non-parallelizable part = 40% of T

Parallelization factor = 2

$$\begin{aligned} T(2) &= 0.4 + (1 - 0.4) / 2 \\ &= 0.4 + 0.6/2 = 0.7 \end{aligned}$$

$$\text{Speed up} = \frac{1}{B + \frac{1-B}{N}} = \frac{1}{0.7} = 10/7 = 1.471$$

$$T(N) = B + (T - B) / N$$

Example 2:

$$B = 0.5$$

$$N = 2$$

- $\text{Speedup} = \frac{1}{B + \frac{1-B}{N}}$
- $\text{SpeedUp} = \frac{1}{B + \frac{1-B}{N}} = \frac{1}{0.5 + \frac{0.5}{2}} = \frac{1}{3/4} = 1.33$

Example 3:

$$B = 0.5$$

$$N = 4$$

- $\text{Speedup} = \frac{1}{B + \frac{1-B}{N}}$
- $\text{SpeedUp} = \frac{1}{B + \frac{1-B}{N}} = \frac{1}{0.5 + \frac{0.5}{4}} = \frac{1}{5/8} = 1.6$

$$T(N) = B + (T - B) / N$$

N = 2

Speedup = 1.33

N=4

Speedup=1.6

N=100

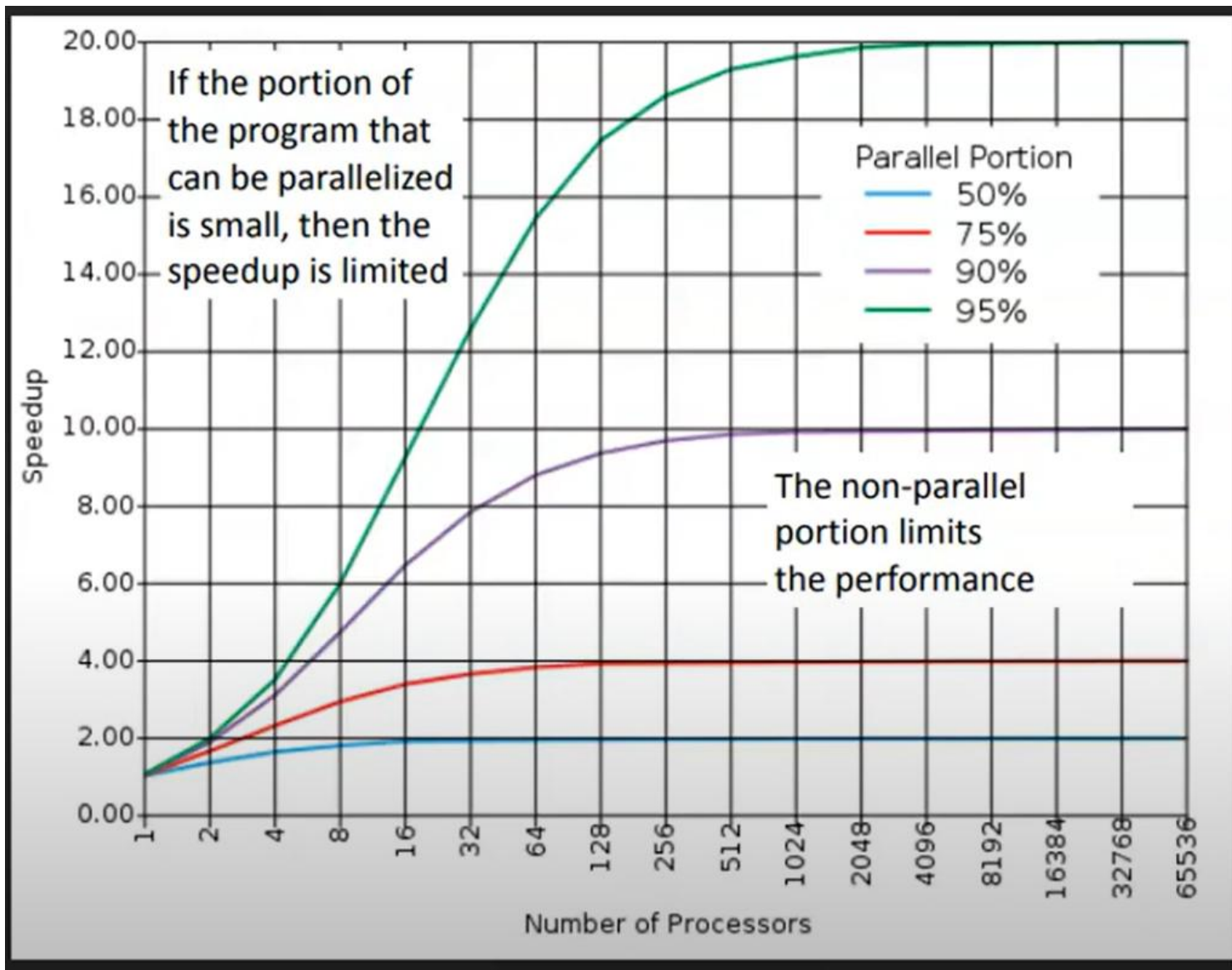
Speedup=1.98

N=1000

Speedup=1.998

N=10000

Speedup=1.9998



Speedup

- **speedup** can be defined as the ratio of execution time for the entire task without using the enhancement and execution time for the entire task u
 - If **Pe** is the performance for the entire task using the enhancement when possible
 - **Pw** is the performance for the entire task without using the enhancement
 - **Ew** is the execution time for the entire task without using the enhancement
 - **Ee** is the execution time for the entire task using the enhancement when possible

$$\text{Speedup} = \frac{P_e}{P_w} \text{ or } \text{Speedup} = \frac{E_w}{E_e}$$

The formula for Amdahl's law is:

$$S = \frac{1}{1 - P + \frac{P}{N}}$$

S is the speedup of the system

P is the proportion of the system that can be improved

N is the number of processors in the system

$$S = \frac{1}{1 - P + \frac{P}{N}}$$

Example : If a system has a single bottleneck that occupies 20% of the total execution time, and we add 4 more processors to the system, the speedup would be:

$$S = 1 / (1 - 0.2 + (0.2 / 5))$$

$$S = 1 / (0.8 + 0.04)$$

$$S = 1 / 0.84$$

$$S = 1.19$$

This means that the overall performance of the system would improve by about 19% with the addition of the 4 processors

Amdahl's law

- How much a computation can be speed up by running part of a program in parallel

Amdhal's Law assumes

- Amdhal's Law assumes that **the size of our problem is fixed**, and given a faster machine, our goal is to run the problem faster.
- In the majority of cases, this is not entirely true. When given a faster machine, we would want to increase the size of the problem or increase the accuracy of the solution.
- If the total amount of time our task ran is 5 seconds, having 3 tasks running in the parallelizable part, we could double the number of tasks running in the parallelizable part from 3 to 6 and our task would still run in 5 seconds.

Gustafson's Law

- **Gustafson's law/(Gustafson-Barsis's law)**
- Addresses shortcomings of the Amdahl's law, which is based on the **assumption** of a **fixed problem size** and **increase** in **resources** does **not improved** the workload.
- It is named after Computer Scientists John L. Gustafson and Edwin H. Barsis.
- Gustafson's Law states that by **increasing** the **problem size**, we can increase the **scalability** and make **use** of **all** the **resources** you have.

Gustafson's Law

- If you apply **N** processors to a task that has a serial fraction **s**, scaling the task takes the same amount of time as before.
- Work is going **N** times fast as 1 processor for the parallel part.
- scaled speedup = $s + p \times N$
= $s + (1-s) \times N$
= $N + (1-N)s$

Gustafson's Law in Action

- Suppose we have a program that is **70% parallel** and **30% sequential**, and we have **10 processors**

- Scaled speedup
$$= N + (1-N)s$$
$$= 10 + (1-10) \times 0.3$$
$$= 7.3$$

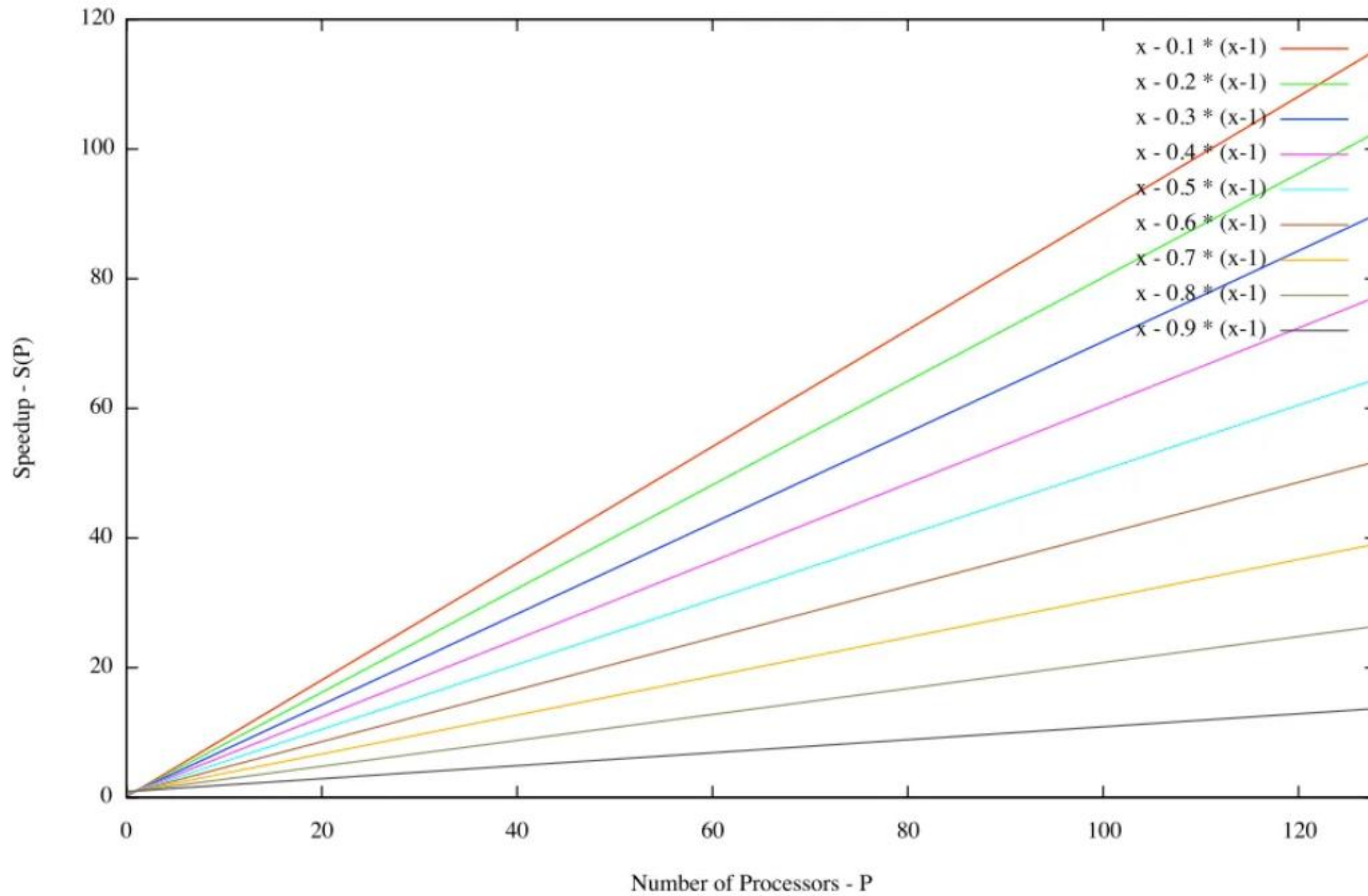
- Suppose for the program above, and we now have **1000** processors

$$\begin{aligned}\text{Scaled speedup} &= N + (1-N)s \\ &= 1000 + (1-1000) \times 0.3 \\ &= 700.3\end{aligned}$$

- If we used Amdahl's Law, the increase in speed up would be from 2.7 to 3.3.

- Speed up = $\frac{1}{B + \frac{1-B}{N}}$ Speed up = $\frac{1}{B + \frac{1-B}{N}} = \frac{1}{0.3 + \frac{0.7}{10}} = 10/3.7 = 2.7$ $\frac{1}{B + \frac{1-B}{N}} = \frac{1}{0.3 + \frac{0.7}{1000}} = 1000/300.7 = 3.3$

Gustafson's Law: $S(P) = P - a \cdot (P - 1)$

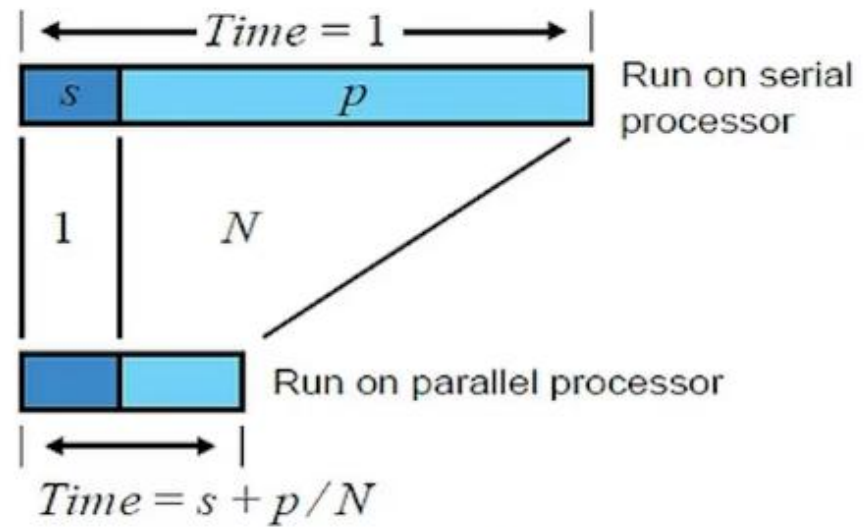


- By **increasing resources** and **problem size**, **speed** will theoretically be a **linear**. The amount of **speed** we get is still be **determined** by the **ratio** of how much of our problem is **parallelized** and how much is **sequential**.
- In summary, how much of a problem can be parallelized and how much not determines the speed of executing a problem. By increasing the problem size and the parallel portion of a problem we can make use of all the resources we have. This also helps in scaling.

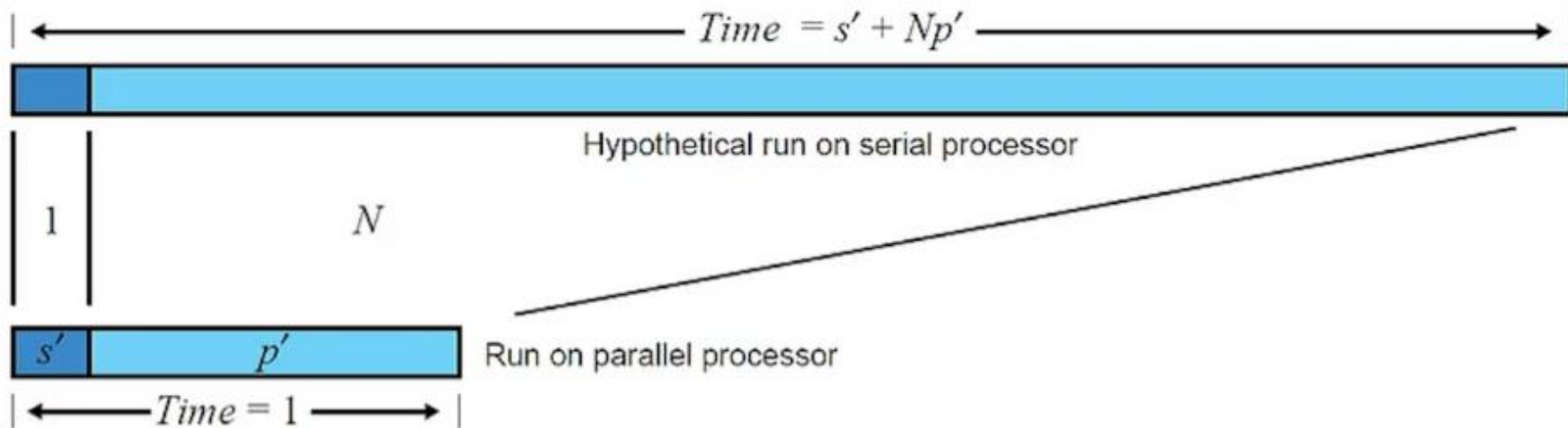
Amdahl's Law and Gustafson's Law

- Amdahl's Law and Gustafson's Law can then be used to estimate speedup. However, neither of these laws considers communication costs or intermediate levels of parallelism.
- Scaled speed up by Gustafson's Law is still limited as the number of processors increases because the communication costs eventually become so high that they negate any benefit gained from increasing the workload size.
- In fact, both laws when applied to modern parallel systems may be inaccurate since communication costs have large effects on speed up

- Amdhal's Law is pessimistic while Gustafson's Law is optimistic
- Looking at parallelism from Amdhal's Law point of view can be discouraging. Gustafson proved that when we increase the amount of work done by the parallel parts, the bottleneck imposed by the sequential parts becomes less malignant.



Fixed-Size Model (Amdahl's Law) [2]



Scaled-Size Model (Gustafson's Law) [2]

Gustafson's Law (aka Gustafson–Barsis's Law)

- if you apply P processors to a task that has serial fraction f , scaling the task to take the same amount of time as before, the speedup is :
- $\text{Speedup} = f + P (1 - f)$
- Gustafson's law addresses the shortcomings of Amdahl's law (which assume fixed problem size) by proposing that programmers can increase size of problems to fully exploit larger computing systems. In a way the law redefines efficiency, due to the possibility that limitations imposed by the sequential part of a program may be countered by increasing the total amount of computation.

Interconnection Networks for Parallel Computers

Why interconnection network?

- Affects the **scalability and cost** of the system
 - How large of a system can you build?
 - How easily can you add more processors?
- Affects **performance and energy efficiency**
 - How fast can processors, caches, and memory communicate?
 - How long are the latencies to memory?
 - How much energy is spent on communication?
- Affects **reliability and security**
 - Can you guarantee messages are delivered or your protocol works?

Interconnection network basic

- **Topology**

- Specifies the way switches are wired
- Affects routing, reliability, throughput, latency, building ease

- **Routing (algorithm)**

- How does a message get from source to destination
- Static or adaptive

- **Buffering and Flow Control**

- What do we store within the routers & links?
 - Entire packets, parts of packets, etc?
- How do we throttle during oversubscription?
- Tightly coupled with routing strategy

Terminology

- Network interface

- Module that connects endpoints (e.g. processors) to network
- Decouples computation/communication

- Link

- Bundle of wires that carry a signal

- Switch/router

- Connects fixed number of input channels to fixed number of output channels

- Channel

- A single logical connection between routers/switches

Terminology

- Node

- A router/switch within a network

- Message

- Unit of transfer for network's clients (processors, memory)

- Packet

- Unit of transfer for network

- Flit

- Flow control digit
- Unit of flow control within network

Terminology

- Network interface

- Module that connects endpoints (e.g. processors) to network
- Decouples computation/communication

- Link

- Bundle of wires that carry a signal

- Switch/router

- Connects fixed number of input channels to fixed number of output channels

- Channel

- A single logical connection between routers/switches

Interconnection Networks for Parallel Computers

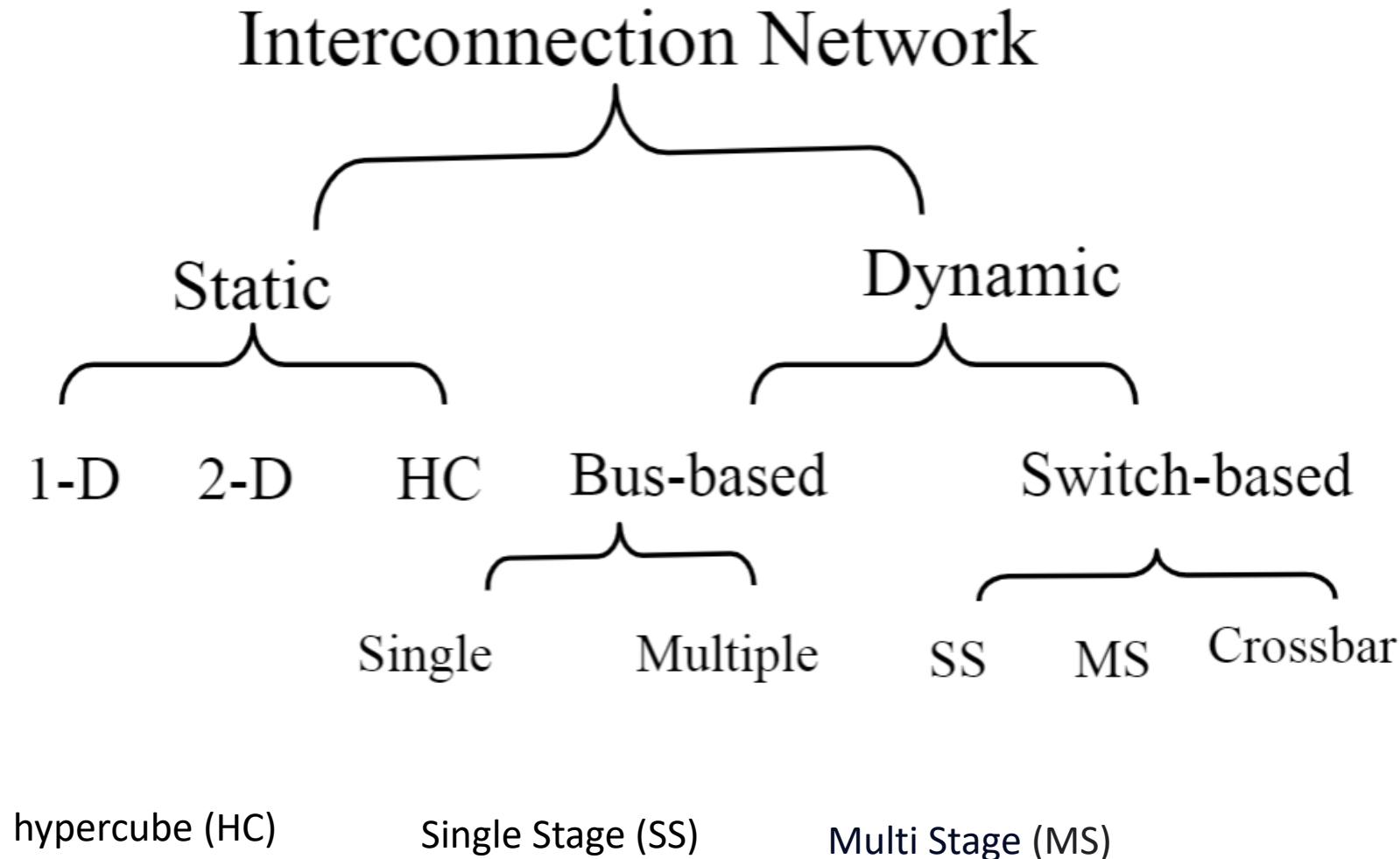
1 Interconnection networks carry data between processors and to memory.

2 Interconnects are made of switches and links (wires, fiber).

3 Interconnects are classified as static or dynamic

4 Static networks consist of point-to-point communication links among processing nodes and are also referred to as direct networks

5 Dynamic networks are built using switches and communication links. Dynamic networks are also referred to as indirect networks



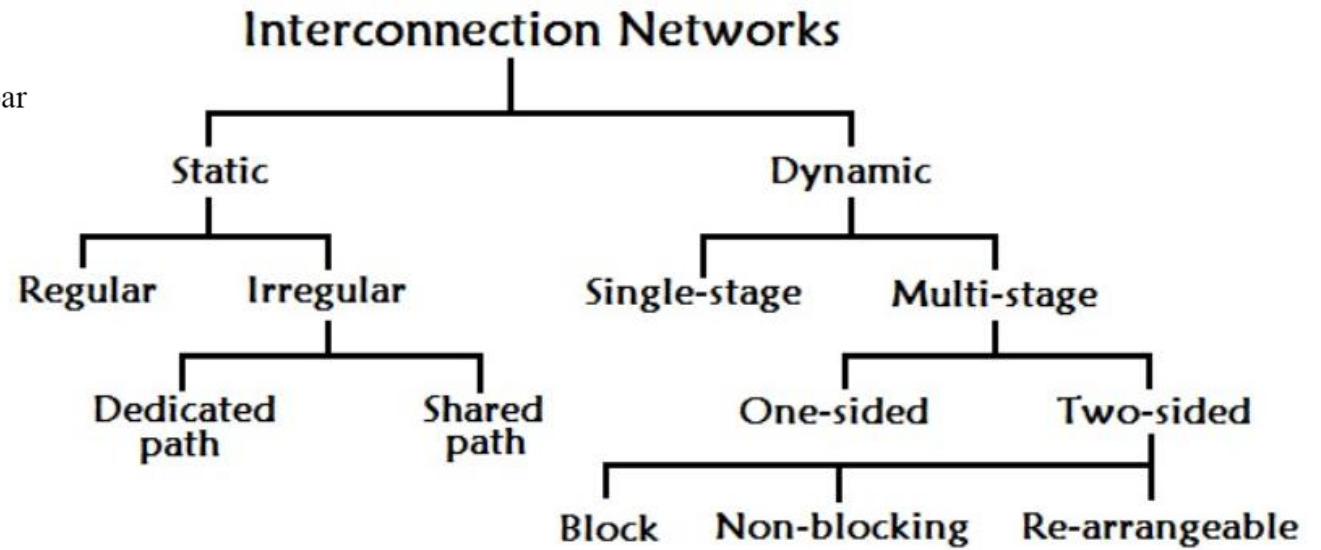
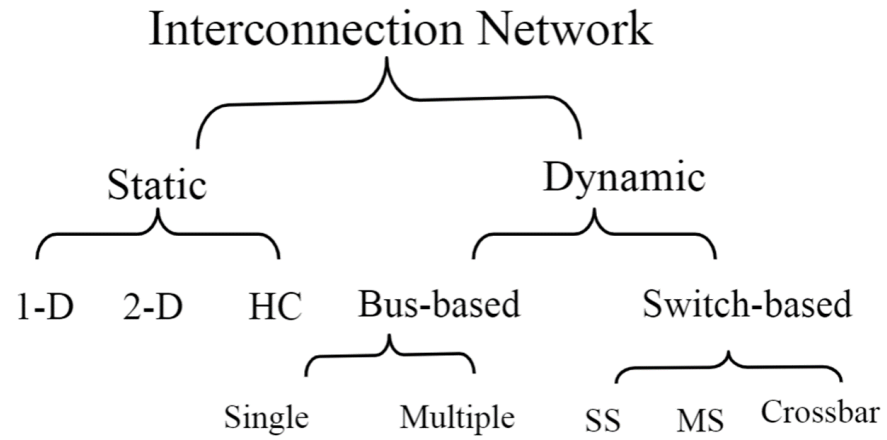
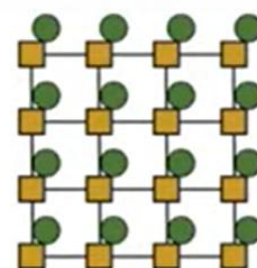
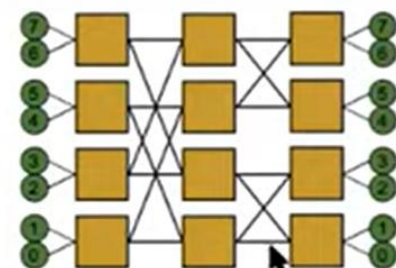
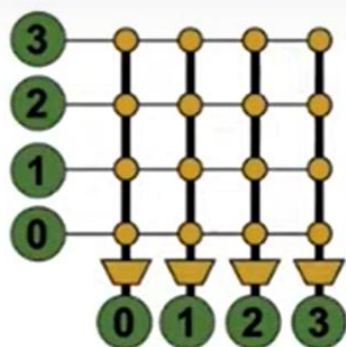
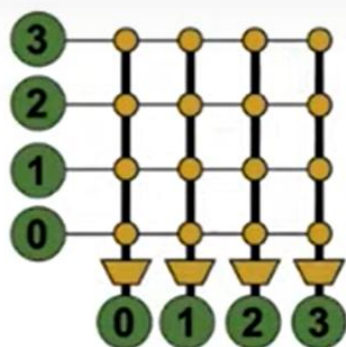
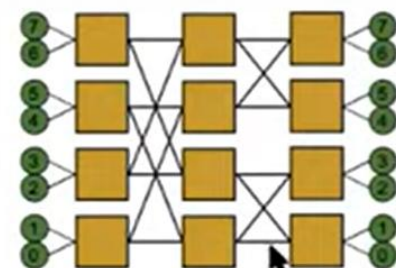
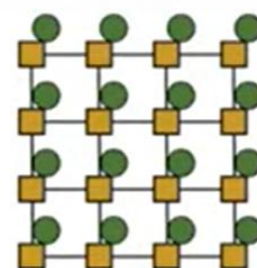


Figure: Classification of Interconnection network

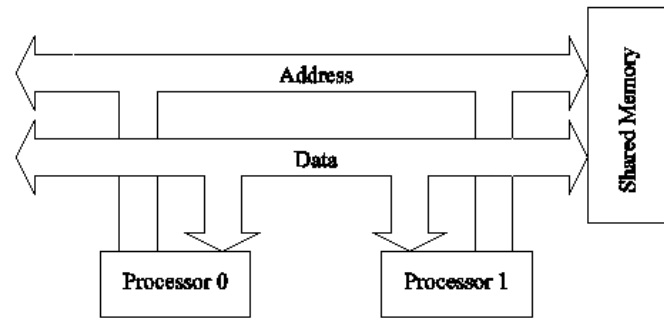


			
Topology	Crossbar	Multistage Logarith.	Mesh
Direct/Indirect	Indirect	Indirect	Direct
Blocking/ Non-blocking	Non-blocking	Blocking	Blocking
Cost	$O(N^2)$	$O(N \log N)$	$O(N)$
Latency	$O(1)$	$O(\log N)$	$O(\sqrt{N})$

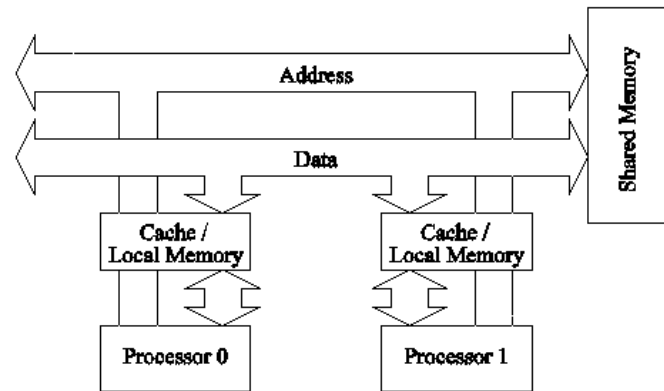
Topology

- Bus (simplest)
- Point-to-point connections (ideal and most costly)
- Crossbar (less costly)
- Ring
- Tree
- Omega
- Hypercube
- Mesh
- Torus
- Butterfly
- ...

Network Topologies: Buses



(a)



(b)

Bus-based interconnects (a) with no local caches; (b) with local memory/caches.

1 simplest and earliest parallel machines used buses.

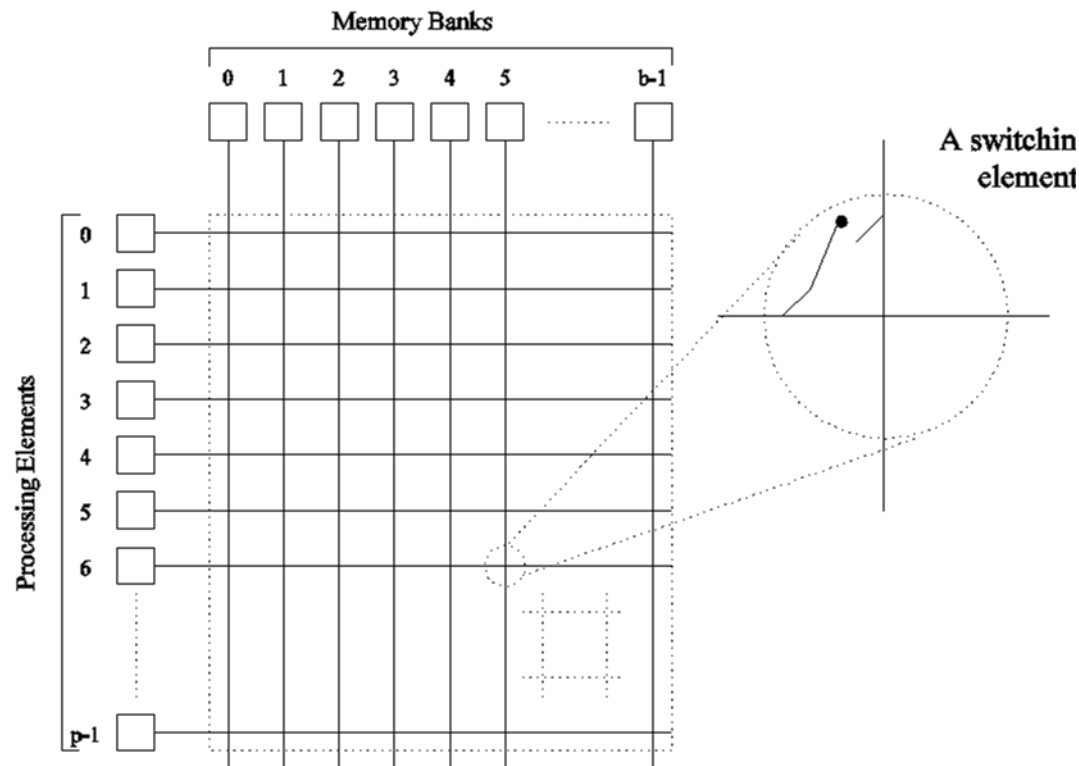
2 All processors access a common bus for exchanging data.

3 The distance between any two nodes is $O(1)$ in a bus. The bus also provides a convenient broadcast media.

4 bandwidth of the shared bus is a major bottleneck

5 Typical bus based machines are limited to dozens of nodes. Sun Enterprise servers and Intel Pentium based shared-bus multiprocessors are examples of such architectures.

Network Topologies: Crossbars



A crossbar network uses an $p \times m$ grid of switches to connect p inputs to m outputs in a non-blocking manner

A completely non-blocking crossbar network connecting p processors to b memory banks.

The cost of a crossbar of p processors grows as $O(p^2)$.

This is generally difficult to scale for large values of p .

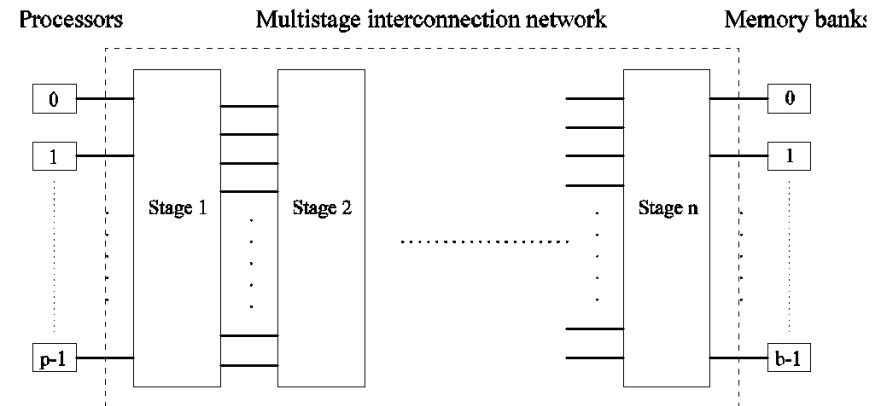
Examples of machines that employ crossbars include the Sun Ultra HPC 10000 and the Fujitsu VPP500.

Blocking & non-blocking

- A blocking switch prevents connecting more than one input.
- A non-blocking switch allows other concurrent connections from inputs to other outputs.

Network Topologies: Multistage Networks

- Crossbars have excellent performance scalability but poor cost scalability.
- Buses have excellent cost scalability, but poor performance scalability.
- Multistage interconnects strike a compromise between these extremes.



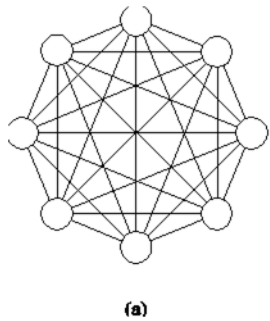
The schematic of a typical multistage interconnection network.

Multistage Omega Network

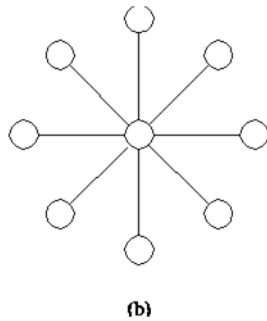
- The most commonly used multistage interconnects
- This network consists of $\log p$ stages, where p is the number of inputs/outputs
- At each stage, input i is connected to output j if:

$$j = \begin{cases} 2i, & 0 \leq i \leq p/2 - 1 \\ 2i + 1 - p, & p/2 \leq i \leq p - 1 \end{cases}$$

Network Topologies: Completely Connected and Star Connected Networks



(a) A completely-connected network of eight nodes;
(b) a star connected network of nine nodes.



Each processor is connected to every other processor. The number of links in the network scales as $O(p^2)$. While the performance scales very well, the hardware complexity is not realizable for large values of p . In this sense, these networks are static counterparts of crossbars.

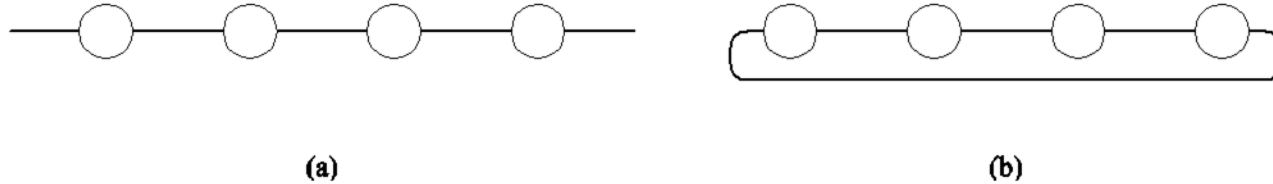
Every node is connected only to a common node at the center. Distance between any pair of nodes is $O(1)$. However, the central node becomes a bottleneck. In this sense, star connected networks are static counterparts of buses.

Network Topologies:

Linear Arrays, Meshes, and k - d Meshes

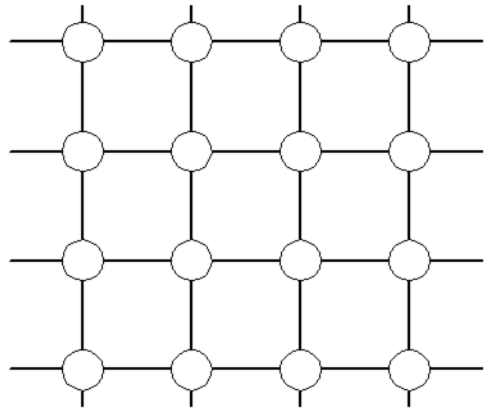
- In a linear array, each node has two neighbors, one to its left and one to its right. If the nodes at either end are connected, we refer to it as a 1-D torus or a ring.
- A generalization to 2 dimensions has nodes with 4 neighbors, to the north, south, east, and west.
- A further generalization to d dimensions has nodes with $2d$ neighbors.
- A special case of a d -dimensional mesh is a hypercube. Here, $d = \log p$, where p is the total number of nodes.

Network Topologies: Linear Arrays

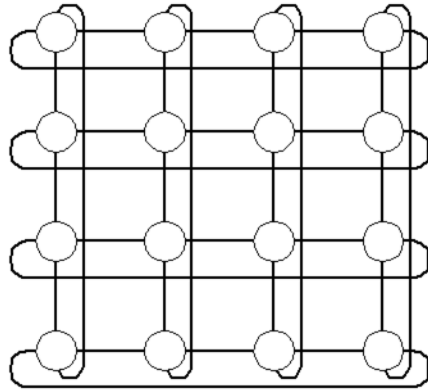


Linear arrays: (a) with no wraparound links; (b) with wraparound link.

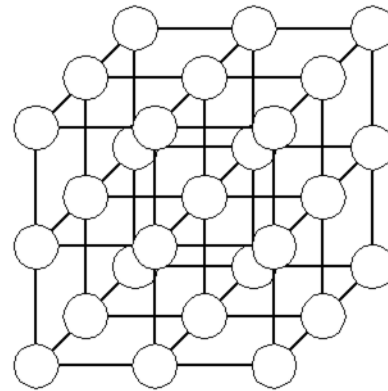
Network Topologies: Two- and Three Dimensional Meshes



(a)



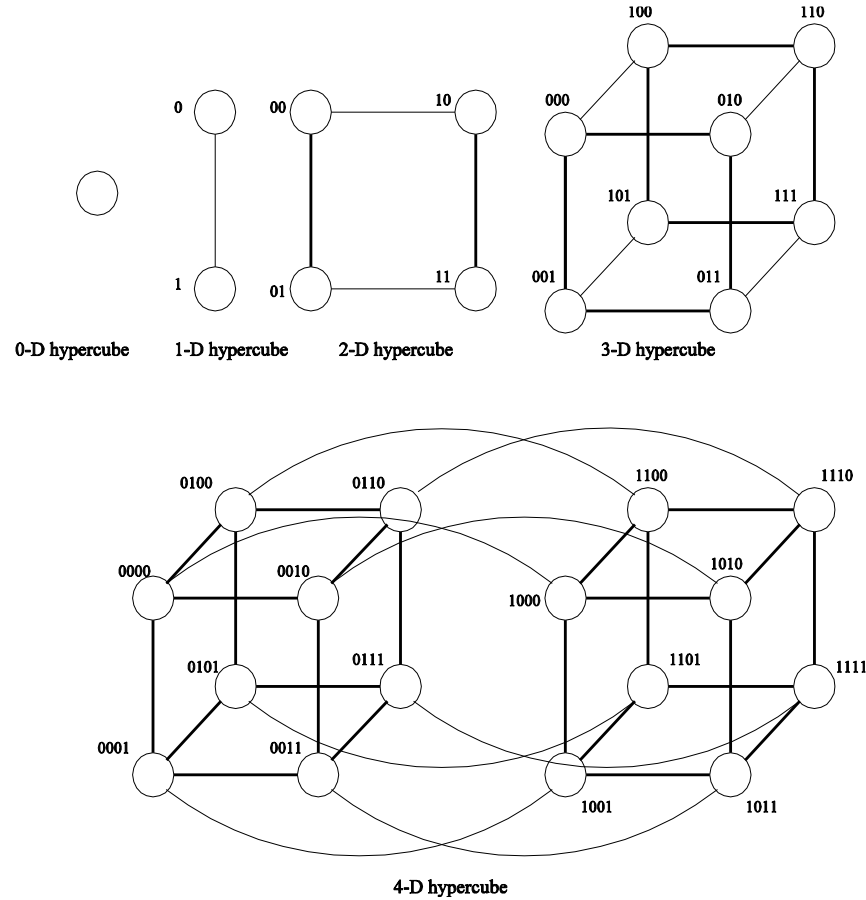
(b)



(c)

Two and three dimensional meshes: (a) 2-D mesh with no wraparound; (b) 2-D mesh with wraparound link (2-D torus); and (c) a 3-D mesh with no wraparound.

Network Topologies: Hypercubes and their Construction



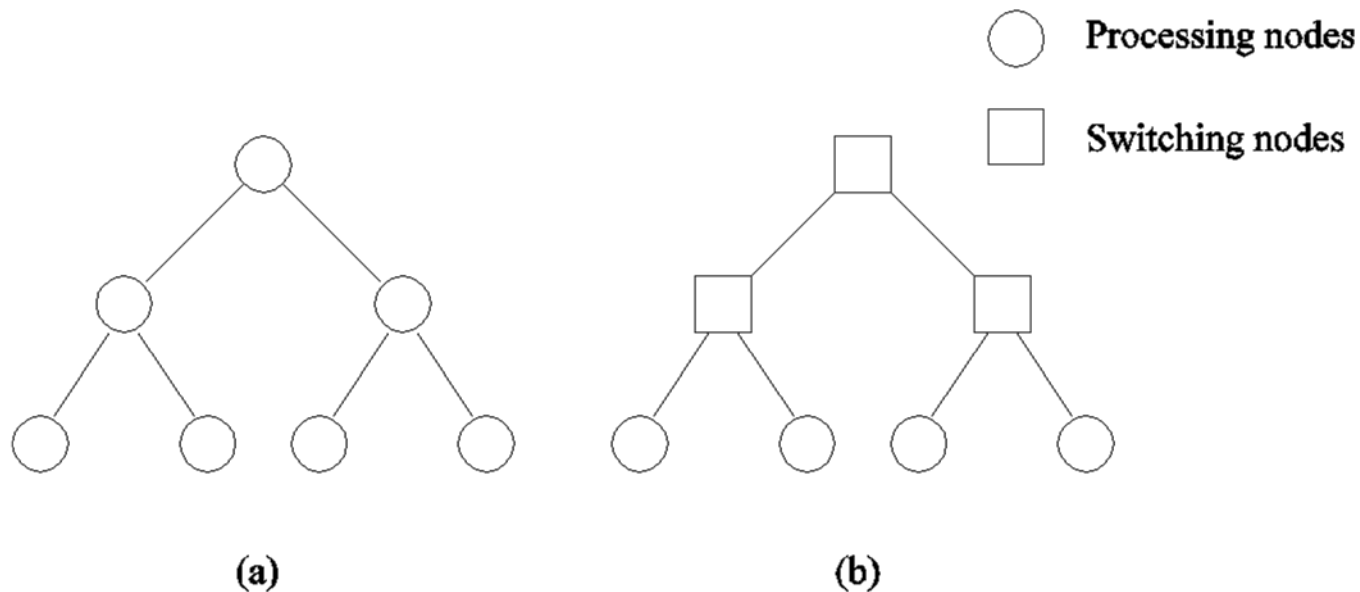
The distance between any two nodes is at most $\log p$.

Each node has $\log p$ neighbors.

The distance between two nodes is given by the number of bit positions at which the two nodes differ.

Construction of hypercubes from hypercubes of lower dimension.

Network Topologies: Tree-Based Networks



Complete binary tree networks: (a) a static tree network; and (b) a dynamic tree network.

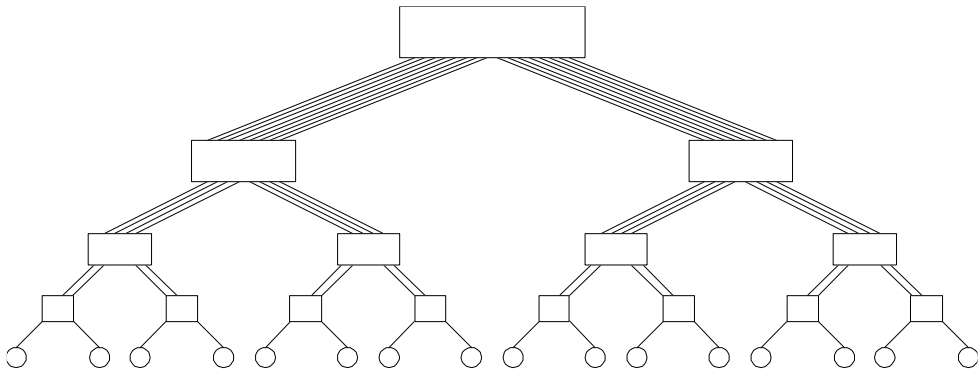
The distance between any two nodes is no more than $2\log p$.

Links higher up the tree potentially carry more traffic than those at the lower levels.

For this reason, a variant called a fat-tree, fattens the links as we go up the tree. Trees can be laid out in 2D with no wire crossings. This is an attractive property of trees.

Network Topologies: Tree Properties

Fat Trees



A fat tree network of 16 processing nodes.

Evaluating Static Interconnection Networks

- *Diameter*: The distance between the farthest two nodes in the network. The diameter of a linear array is $p - 1$, that of a mesh is $2(\sqrt{p} - 1)$, that of a tree and hypercube is $\log p$, and that of a completely connected network is $O(1)$.
- *Bisection Width*: The minimum number of wires you must cut to divide the network into two equal parts. The bisection width of a linear array and tree is 1 , that of a mesh is $\sqrt{p}$, that of a hypercube is $p/2$ and that of a completely connected network is $p^2/4$.
- *Cost*: The number of links or switches (whichever is asymptotically higher) is a meaningful measure of the cost. However, a number of other factors, such as the ability to layout the network, the length of wires, etc., also factor in to the cost.

Network	Diameter	Bisection Width	Arc Connectivity	Cost (No. of links)
Completely-connected				
Star				
Complete binary tree				
Linear array				
2-D mesh, no wraparound				
2-D wraparound mesh				
Hypercube				
Wraparound k -ary d -cube				

Evaluating Static Interconnection Networks

Network	Diameter	Bisection Width	Arc Connectivity	Cost (No. of links)
Completely-connected	1	$p^2/4$	$p - 1$	$p(p - 1)/2$
Star	2	1	1	$p - 1$
Complete binary tree	$2 \log((p + 1)/2)$	1	1	$p - 1$
Linear array	$p - 1$	1	1	$p - 1$
2-D mesh, no wraparound	$2(\sqrt{p} - 1)$	$\sqrt{p}$	2	$2(p - \sqrt{p})$
2-D wraparound mesh	$2\lfloor \sqrt{p}/2 \rfloor$	$2\sqrt{p}$	4	$2p$
Hypercube	$\log p$	$p/2$	$\log p$	$(p \log p)/2$
Wraparound k -ary d -cube	$d\lfloor k/2 \rfloor$	$2k^{d-1}$	$2d$	dp

Network	Diameter	Bisection Width	Arc Connectivity	Cost (No. of links)
Crossbar				
Omega Network				
Dynamic Tree				

Evaluating Dynamic Interconnection Networks

Network	Diameter	Bisection Width	Arc Connectivity	Cost (No. of links)
Crossbar	1	p	1	p^2
Omega Network	$\log p$	$p/2$	2	$p/2$
Dynamic Tree	$2 \log p$	1	2	$p - 1$

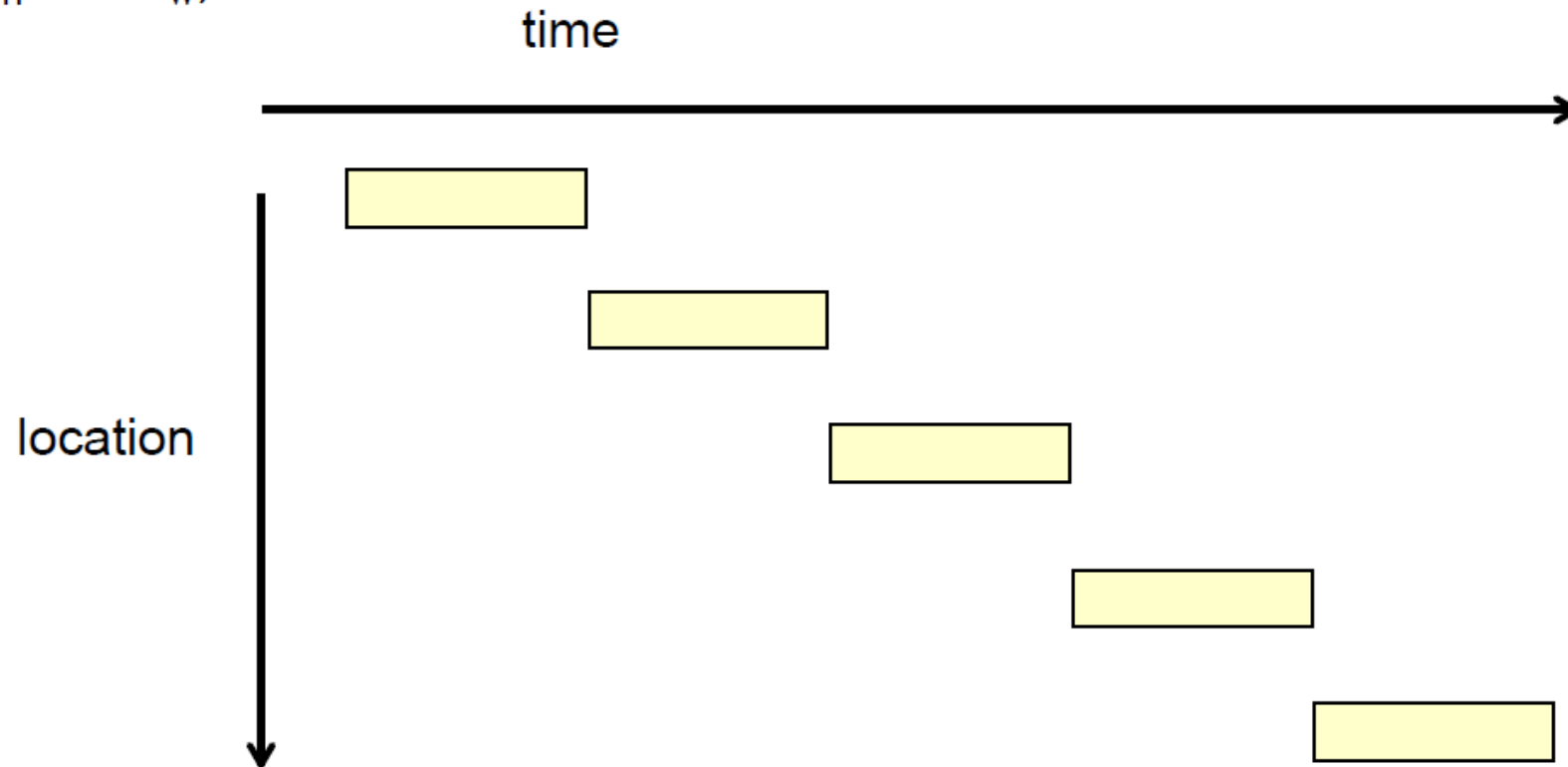
Evaluating Dynamic Interconnection Networks

Network	Diameter	Bisection Width	Arc Connectivity	Cost (No. of links)
Crossbar	1	p	1	p^2
Omega Network	$\log p$	$p/2$	2	$p/2$
Dynamic Tree	$2 \log p$	1	2	$p - 1$

Store-and-forward routing

- flow-control mechanism at message or packet level
- packets are transferred one link at a time
- large buffers, high latency
- cost

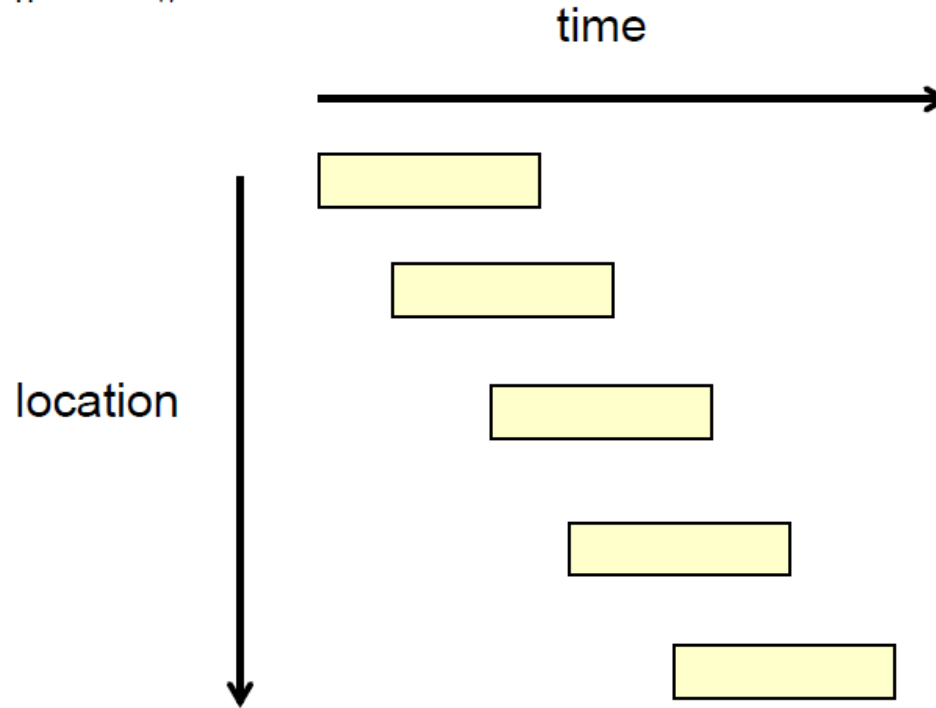
$$t_{SF} = t_s + (t_h + m t_w) h$$



Cut-through routing

- flow control is per-link and payload transmission is pipelined
- message spread out across multiple links in the network
- small buffers, low latency
- cost

$$t_{CT} = t_s + ht_h + mt_w$$



Further reference

- https://www.youtube.com/watch?v=P_57FZSnrO4